Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3695053.3731041

RESEARCH-ARTICLE

OptiPIM: Optimizing Processing-in-Memory Acceleration Using Integer Linear Programming

JIANTAO LIU, Swiss Federal Institute of Technology, Zurich, Zurich, ZH, Switzerland

MINXUAN ZHOU, Illinois Institute of Technology, Chicago, IL, United States

YUE PAN, University of California, San Diego, San Diego, CA, United States

CHIEN-YI YANG, University of California, San Diego, San Diego, CA, United States

LANA JOSIPOVIĆ, Swiss Federal Institute of Technology, Zurich, Zurich, ZH, Switzerland

TAJANA ROSING, University of California, San Diego, San Diego, CA, United States

Open Access Support provided by:

University of California, San Diego

Swiss Federal Institute of Technology, Zurich

Illinois Institute of Technology

PDF Download
3695053.3731041.pdf
04 March 2026
Total Citations: 0
Total Downloads: 10070

Published: 20 June 2025

Citation in BibTeX format

ISCA '25: Proceedings of the 52nd Annual International Symposium on Computer Architecture
June 21 - 25, 2025
Tokyo, Japan

Conference Sponsors:
SIGARCH

OptiPIM: Optimizing Processing-in-Memory Acceleration Using Integer Linear Programming

Jiantao Liu*
ETH Zürich
Zürich, Switzerland
jianliu@ethz.ch

Minxuan Zhou*
Illinois Institute of Technology
Chicago, USA
mzhou26@iit.edu

Yue Pan
University of California, San Diego
La Jolla, USA
yup014@ucsd.edu

Chien-Yi Yang
University of California San Diego
La Jolla, USA
chy036@ucsd.edu

Lana Josipović
ETH Zürich
Zürich, Switzerland
ljosipovic@ethz.ch

Tajana Rosing
UCSD
La Jolla, USA
tajana@ucsd.edu

Abstract

Processing-in-memory (PIM) accelerators provide superior performance and energy efficiency to conventional architectures by minimizing off-chip data movement and exploiting extensive internal memory bandwidth for computation. However, efficient PIM acceleration requires careful software-hardware mapping that transforms application algorithms into PIM operations and data layout. Unfortunately, existing PIM accelerators adopt manually tuned heuristics or exhaustive search to determine the mappings on PIM accelerators, leading to under-optimized performance and/or long optimization time. In this work, we propose OptiPIM, a novel optimization framework based on Integer Linear Programming (ILP) to efficiently generate the optimal mapping for data-intensive applications on PIM accelerators. The proposed framework adopts a PIM-friendly mapping representation with accurate cost modeling and a concise description of the entire design space, allowing us to formulate an efficient and effective ILP problem and optimize the mapping on PIM architectures. We implement OptiPIM in the open-source MLIR framework, enabling OptiPIM to generate optimized mappings for PyTorch workloads on PIM accelerators. We evaluate widely used machine learning workloads on two state-of-the-art PIM accelerators. Our experiments show that OptiPIM can generate optimal mappings within 4 minutes. Mappings generated by OptiPIM are at least 1.9× faster than those generated by heuristics.

CCS Concepts

• **Hardware** → **Emerging technologies**; • **Computing methodologies** → **Modeling and simulation**; *Machine learning*.

Keywords

Processing in memory, mapping optimization, modeling.

*Both authors contributed equally to this research.

This work is licensed under a Creative Commons Attribution 4.0 International License.
ISCA '25, Tokyo, Japan
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1261-6/25/06
https://doi.org/10.1145/3695053.3731041

ACM Reference Format:
Jiantao Liu, Minxuan Zhou, Yue Pan, Chien-Yi Yang, Lana Josipović, and Tajana Rosing. 2025. OptiPIM: Optimizing Processing-in-Memory Acceleration Using Integer Linear Programming. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25), June 21–25, 2025, Tokyo, Japan.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3695053.3731041

1 Introduction

Processing-in-memory (PIM) is promising in tackling the memory wall problem [13] of conventional systems by exploiting large internal bandwidth for computing and minimizing inefficient data movements [1, 11, 25, 35–38, 56]. As an unconventional computing paradigm, PIM enforces strict data layout requirements, making it particularly challenging to map data-intensive operations with complex indexing patterns onto PIM architectures. This challenge is particularly evident in Deep Neural Networks (DNNs), which involve *operators* such as convolution, fully-connected layers, and batched matrix multiplications. However, existing PIM accelerators only rely on heuristics or exhaustive/random search-based algorithms to generate mappings for such operators, leading to sub-optimal mappings and/or long optimization times.

Several recent works have proposed white-box mapping frameworks for conventional ASIC-based DNN accelerators [20, 23, 52]. However, these methods do not directly apply to PIM accelerator mapping. First, prior ASIC mapping representations [20, 23, 52] cannot effectively describe different fine-grained data layouts required by PIM computation. Specifically, prior ASIC mapping frameworks only consider computation and data access patterns when generating the mapping, based on the assumption that the accelerator can load arbitrary data for computations across the memory hierarchy. In contrast, PIM accelerators enforce specific data layout regulations for a mapping—data in memory must strictly align with the computation. Such a layout enforcement can result in significant data replication for different computations (as we will detail in Section 3 and Figure 2), leading to a high data movement cost and a large memory footprint. Second, the modeling algorithms in prior ASIC mapping frameworks [20, 23, 52] cannot accurately estimate the real costs of PIM acceleration. For example, prior frameworks use loop bound products to estimate the data size required for each mapping. However, this upper-bound modeling significantly

overestimates the actual data size for each mapping. Such an overestimation leads to large errors in estimating the PIM layout-related cost and memory footprint, leading to sub-optimal mappings.

In this work, we propose OptiPIM, a framework that exploits an exact optimization technique, integer linear programming (ILP), to generate mappings for data-intensive applications on PIM accelerators. Adopting ILP enables OptiPIM to efficiently explore the complex design space of PIM mappings and generate optimal mappings that significantly outperform prior-art heuristics. We first propose a representation to describe mappings on a general PIM architectural model, which supports various emerging PIM architectures, such as bit-serial processing using memory (PuM) [16, 45] and near-bank processing (NBP) [35]. The proposed mapping representation enables comprehensive and concise modeling for computation scheduling, data access, and data layout in PIM acceleration. Specifically, we introduce several layout-aware abstraction levels into the conventional nested loop representation and consider all possible data indexing schemes to comprehensively represent the design space with a bounded number of variables, enabling efficient and effective optimization. Building on this concise representation, we formulate PIM mapping as an ILP problem that jointly models the application's algorithmic structure and the underlying hardware. The ILP incorporates a comprehensive set of costs and constraints, each highly configurable, so that the same formulation can generate optimized mappings across diverse PIM architectures. In addition, we introduce a new cost estimation technique that computes data sizes more accurately than previous ASIC-oriented mapping frameworks.

We implement OptiPIM in the open-source MLIR framework [34]. Our flow starts from a high-level PyTorch computational graph, which is then lowered to the MLIR linalg dialect via Torch-MLIR [39], and translated to nested affine loops for ILP modeling and optimization. Finally, OptiPIM produces the mapping, including data allocation and memory operation trace, for the target PIM accelerator [16, 35]. In summary, our contributions include:

- We propose OptiPIM, a framework that formulates the mapping of data-intensive applications on PIM accelerators as an ILP problem. OptiPIM systematically identifies critical costs and constraints inherent to PIM architectures and integrates them into accurate cost modeling algorithms, which improves the quality of the mapping solution.
- OptiPIM adopts a novel mapping representation that can concisely describe the entire mapping design space of a general PIM architecture using a layout-aware nested loop structure and flexible indexing.
- We comprehensively compare the quality of generated mappings of OptiPIM and existing mapping methods on two state-of-the-art PIM accelerators. Our experiments show OptiPIM outperforms prior mapping methods by at least 1.9×, with a maximum of 4-minute optimization runtime.

2 Background

2.1 Processing-in-Memory Accelerators

Processing-in-Memory (PIM) accelerators use different memory technologies, including DRAM [9, 16, 24, 38, 45], SRAM [11, 26], and ReRAM [1, 25, 56]. We here focus on high-throughput digital PIM, including bit-serial processing using memory (PuM) [11, 16, 25, 38, 45] and near-bank processing (NBP) [18, 32, 33, 35, 36].

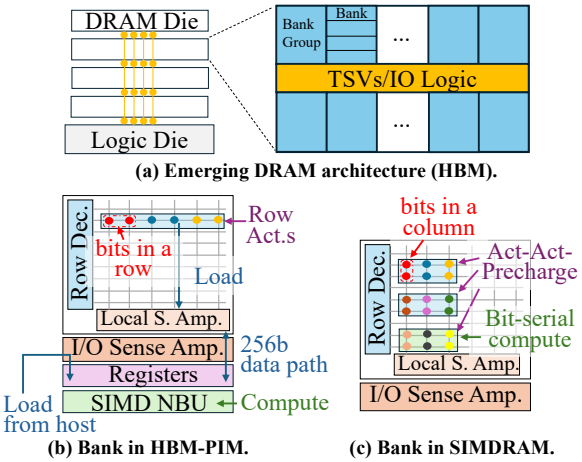


Figure 1: PIM architectures and data organizations.

Figure 1(a) shows high-bandwidth memory (HBM) [44], which is used as the base architecture in this paper. Figures 1(b) and (c) show the bank architecture and basic computing paradigm in HBM-PIM [35] and SIMDRAM [16], which are the state-of-the-art NBP and PuM technologies, respectively. HBM-PIM integrates near-bank compute logic connected to the internal DRAM data path. It consists of general and scalar registers and a near-bank unit (NBU) with multiple floating-point units that support single-instruction-multiple-data (SIMD) computations by utilizing the internal data link (i.e., 256b). The registers can store vector or scalar data for computation. HBM-PIM supports an all-bank operation mode, which operates on a specific DRAM column or register across all banks.

Unlike HBM-PIM, SIMDRAM [16] (and other PuM accelerators [11, 12, 38, 45]) exploit the intrinsic DRAM operations to manipulate memory cell rows. In particular, it exploits multi-row activations and a sequence of activate-activate-precharge (AAP) operations to achieve logic functions (e.g., MAJ, NOT, etc.). These basic logic functions then support arbitrary operations, such as multi-bit addition and multiplication, on data stored in the same rows in parallel. Bit-level manipulation requires SIMDRAM to place the data bits vertically and process computations in a bit-serial fashion.

2.2 PIM-Specific Data Layout Enforcement

Compared to conventional ASIC accelerators, PIM accelerators feature several PIM-specific characteristics due to the need to align data in memory for computation. Such layout enforcement applies to both NBP and PuM architectures. We use HBM-PIM [35] and SIMDRAM [16] as examples for illustration.

HBM-PIM [35] enforces data layout because its SIMD registers directly read contiguous columns from either a DRAM row or the IO path from the host. To support HBM-PIM computations, programmers need to 1) place stationary data (e.g., DNN weights) in DRAM rows in the same way as the SIMD operands and/or 2) write dynamic data (e.g., DNN input feature maps), packed in SIMD format for each computation, to near-bank registers. The results

```
for (o = 0; o < 8; o++):
  for (f = 0; f < 6; f++):
    output[o] += input[o+f] * filter[f]
```

(a) Example 1D convolution

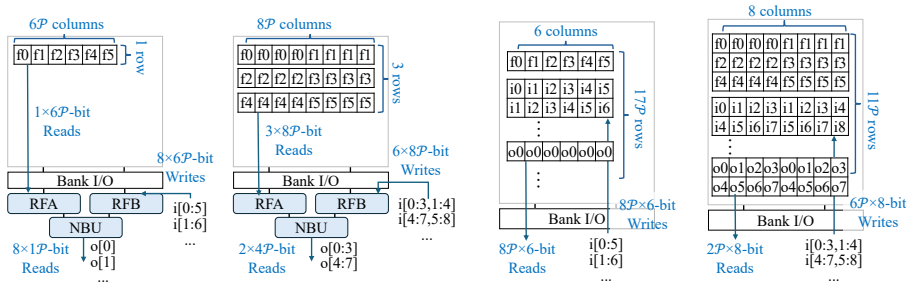
Mapping 1

Time	Op1	Op2	Result
0	f[0:5]	i[0:5]	o[0]
1	f[0:5]	i[1:6]	o[1]
...	…	…	…
7	f[0:5]	i[7:12]	o[7]

Mapping 2

Time	Op1	Op2	Result
0	f[0]*4,f[1]*4	i[0:3,1:4]	o[0:3]
1	f[0]*4,f[1]*4	i[4:7,5:8]	o[4:7]
...	…	…	…
5	f[4]*4,f[5]*4	i[8:11,9:12]	o[4:7]

(b) Computation schedule for 2 mappings



(c) Mapping 1 (left) and Mapping 2 (right) on HBM-PIM

(d) Mapping 1 (left) and Mapping 2 (right) on SIMDRAM

Figure 2: PIM-specific mapping problem on HBM-PIM [35] and SIMDRAM [16] for the 1D convolution operator.

(e.g., multiply-and-accumulate) of a SIMD operation either stay in registers or are transferred to the host for subsequent operations.

Compared to HBM-PIM [35], the data layout enforcement is more obvious in SIMDRAM [16]. To support PuM operations that directly operate on DRAM cells, SIMDRAM must align all operands in DRAM rows with a vertical bit-precision layout, requiring conventional DRAM writes for non-stationary data. The basic setting of SIMDRAM only supports element-wise arithmetic. Therefore, if the partial results of a dot product are not aligned in the memory, we need to send all the partial results to the host for reduction. This leads to various PuM architectures that add special logic to process partial product reduction in memory [11, 45, 66].

2.3 Mapping Generation

Applications such as machine learning and computer graphics rely on data-intensive operators to process multi-dimensional data structures (tensors) using nested loops. Nested loops can represent various iteration patterns (e.g., orders of computation) by decomposing and reordering loop levels, leading to a large design space for mapping the corresponding operator onto hardware. Various recent works have extended the nested loop representation to denote hardware design choices, such as spatial computation, buffering strategies, and data access patterns [1, 20, 21, 23, 47, 64]. Mapping, in this context, aims to find the detailed nested loop representation with specific loop decomposition, ordering, and hardware-related tags that provide the best performance. Prior works utilize exhaustive search with manually-specified cosntraints [1, 21, 47, 64] or ILP [20, 23, 52] for the optimization. In this work, we also target operator mapping described by nested loops. In Section 3, we illustrate specific challenges of mapping onto PIM accelerators, which existing methods fail to address.

3 Motivation

3.1 Data Layout Critically Impacts Performance

Data layout enforcement results in additional design complexity in resource allocation, computation, and data movement on PIM architectures. Figure 2 shows two different mappings of a 1D convolution (Figure 2(a)) and the corresponding data layouts on NBP (HBM-PIM [35]) and PuM (SIMDRAM [16]) PIM accelerators.

Figure 2(b) shows the computation scheduling of two sample mappings, where each row indicates a time step of a SIMD operation using filter (Op1) and input (Op2) to compute output (Result). Specifically, in Mapping 1, each time step processes all 6 partial sums of 1 output index in parallel. Therefore, Mapping 1 exhibits a weight-stationary behavior where the weight vector (convolution filter) can be reused in all time steps for different input vectors. In Mapping 2, each time step computes 8 partial sums in parallel, 2 partials for each of the 4 output indices. Therefore, Mapping 2 requires 2 steps less than Mapping 1. However, Mapping 2 requires data replication in filter vectors, and each can only be reused once. In both mappings, each time step requires a different input pattern. This leads to zero reuse across different time steps because NBP and PuM accelerators lack the functionality of arbitrary reshuffling inside the memory, necessitating input loading for each time step.

Figure 2(c) shows the data layout and various costs of Mapping 1 and 2 on HBM-PIM [35]. As shown in the figure, Mapping 1 and 2 require 1 row/$6\mathcal{P}$ columns and 3 rows/$8\mathcal{P}$ columns in HBM-PIM, respectively, where $\mathcal{P}$ denotes the data precision. The latency costs of each mapping on HBM-PIM include reading filter values from bank to registers (RFA), writing input values from the host to registers (RFB), computing in the near-bank unit (NBU), and reading output values from registers to the host. Except for the computation cost, which depends on the computation scheduling, all other costs are highly dependent on the data layout inside the memory. For example, as shown in the left side of Figure 2(c), Mapping 1 on HBM-PIM first reads 1 row of 6 filter values, each with $\mathcal{P}$ bits, from the bank to RFA ($1 \times 6\mathcal{P}$-bit reads). Then, the host writes 8 input vectors, each with 6 $\mathcal{P}$-bit values, to RFB ($8 \times 6\mathcal{P}$-bit writes). After all computations, the host can read the computed outputs (o[0] - o[7]) from the NPU ($8 \times 1\mathcal{P}$-bit reads). As shown in the right side of Figure 2(c), Mapping 2 has a different layout, which reads 3 rows of $8\mathcal{P}$-bit values from the bank to RFA, and writes 6 rows of $8\mathcal{P}$-bit values to RFB. The NBU then produces 2 rows of 4 $\mathcal{P}$-bit values, which are sent to the host through reads.

Figure 2(d) shows Mapping 1 and 2 on SIMDRAM [16]. Mapping 1 and 2 require $17\mathcal{P}$ rows/6 columns and $11\mathcal{P}$ rows/8 columns, respectively, due to the bit-serial data layout. The data loading costs of SIMDRAM are similar to those of HBM-PIM, which depends on the detailed data layout. However, due to the lack of support for

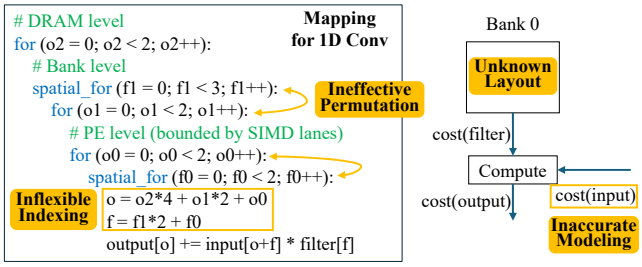


Figure 3: The issues of ASIC mapping representations [23, 47] on PIM acceleration (example mapping for Figure 2(a)).

reduction in SIMDRAM, we need to read out all partial sums after the computation (e.g., $8\mathcal{P}$ rows of 6 bits for Mapping 1). SIMDRAM has much higher computation costs due to bit-serial processing. Specifically, addition and multiplication require $O(\mathcal{P})$ and $O(\mathcal{P}^2)$ activate-activate-precharge (AAP) operations, respectively, where each AAP takes around 50 ns [16]. Such a significant difference necessitates different considerations of the computation cost on the two PIM architectures.

These examples show the importance of data layout on PIM accelerators. The corresponding mapping optimizations must consider the fine-grained data layout, which determines critical costs, including computation, data movement, and memory footprint.

3.2 ASIC Mappings Are Insufficient for PIM

Existing ASIC mapping frameworks [20, 23, 52] do not consider the fine-grained data layout of computation, resulting in several issues for the PIM mapping problem, as illustrated in Figure 3.

3.2.1 Mapping Representation. **Prior mapping representations cannot effectively describe the valid design space of the tensor-program–to-PIM mapping problem.** Specifically, existing ASIC-based mapping representations assume a hierarchical memory architecture and focus only on the data access patterns across the memory/cache hierarchy and the computation unit. On PIM architectures, such a mapping representation can only describe high-level computation scheduling across different processing units, not the fine-grained data layout within the memory (i.e., inside the banks). Figure 3 shows an example of using an ASIC-based representation [23] to represent a mapping on a PIM accelerator that distributes filters across multiple SIMD lanes (f0) in different banks (f1). However, the detailed data layout within each bank is not captured by the representation. Furthermore, the ASIC representation allows flexible loop permutations (e.g., swapping f1 and o1 in the example). On PIM architectures, such loop permutations do not always result in a different mapping because 1) only the relative order between spatial loops determines the distribution of computations across memory components, and 2) only the relative order between temporal loops determines the sequence of computation in each memory component. Allowing such loop transformations introduces numerous redundant mappings.

Another issue is that existing mapping representations only consider stride-1 indexing (see Figure 3, inflexible indexing), which always places data elements with consecutive indices in the same row/column. The inflexibility of the data indexing function unnecessarily reduces the design space. It becomes more serious in PIM accelerators than ASIC accelerators because the data indexing function also determines the fine-grained data layout in memory. Thus, the mapping representation used by existing ASIC accelerators cannot be effectively applied to PIM accelerators. In Section 4, we propose a representation that efficiently and effectively describes the valid design space for mapping generation on PIM accelerators.

3.2.2 Cost Modeling and Problem Formulation. **The cost modeling used by prior optimization frameworks [23] for evaluating mappings on ASIC accelerators is not only insufficient but also inaccurate for PIM accelerators.** Specifically, prior ASIC frameworks only count the data accesses and transfers in hierarchical memory, without considering layout-related costs. As shown in Figure 2, all data movement costs, including reading and writing operations for different tensors, depend on the detailed data layout in memory. Therefore, PIM mapping needs accurate layout-aware cost modeling in the problem formulation. In Section 5.3, we show that a prior ASIC cost estimation method can introduce on average 70% error when estimating the required data size for PIM computation. We therefore propose an accurate PIM cost model that introduces less than 3% error.

3.3 Existing PIM Mappings Are Inefficient

Unlike ASIC accelerators, the mapping methods used in existing PIM accelerators still rely on heuristics or exhaustive/random searches [7, 35, 38, 65]. For example, HBM-PIM [35] adopts a heuristic to distribute the matrix operand of a vector-matrix multiplication (e.g., weight matrix of a fully connected layer in DNN) across multiple SIMD lanes on multiple DRAM banks and compute partial results of output values in parallel. When adopting this heuristic to convolution (HBM-PIM does not illustrate how to map a convolution), the mapping is similar to Mapping 1 in Figure 2(c), which distributes the filters of convolution for SIMD processing. Such a heuristic has two issues. First, it may not fully exploit all available SIMD lanes when the weight matrix or convolution filter is neither large enough nor perfectly factorizable. Second, it does not consider the trade-offs between parallelism and layout-related costs, as more parallelism introduces more data replication. While heuristics cannot guarantee optimal results, exhaustive search methods [65] require a long execution time to find the optimal mapping. No existing work proposed an efficient methodology to comprehensively explore the intricate trade-off space in PIM mapping.

4 OptiPIM Mapping Representation

This section describes our PIM mapping representation that can effectively express all aspects of the PIM mapping problem.

4.1 General PIM Hardware Model

We first illustrate the PIM architecture model used by OptiPIM, which can be configured to represent different PIM architectures.

4.1.1 High-Level Abstraction. Figure 4(a) shows the high-level abstraction of a general PIM architecture consisting of PIM units (PUs), which are the basic components for conducting computations in the PIM-enabled memory. Each PU has its local memory partition

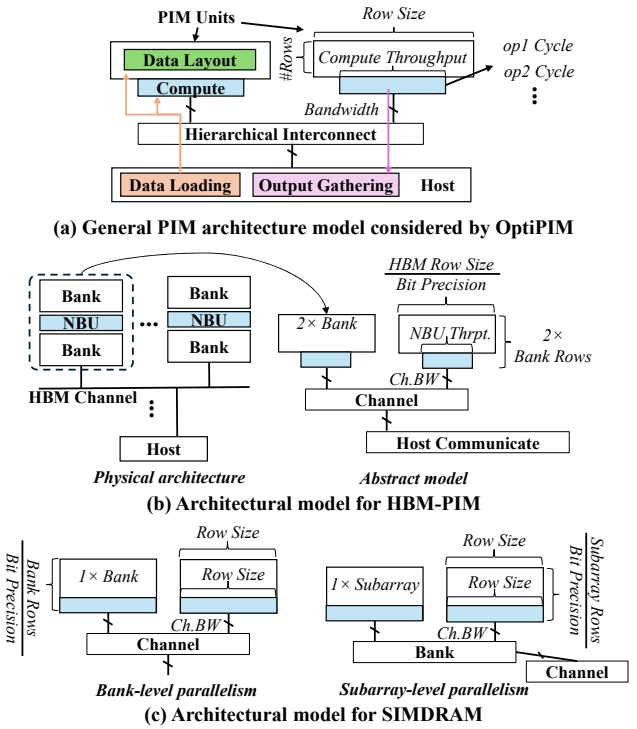


Figure 4: OptiPIM's PIM–architecture abstraction and its instantiations for HBM-PIM [35] and SIMDRAM [16].

for storing data used for computations. A PU has two basic architectural parameters, *Row_Size* and *#Rows*, to denote the 2D dimension of the local memory partition (e.g., DRAM bank). In addition to the architectural parameters, the PU configuration also includes parameters to denote the cost of basic operations (e.g., latency) and computation throughput (e.g., SIMD width, data loading burst width, etc.). Above the PU level, our general PIM hardware model incorporates a hierarchical architecture that organizes PUs through multi-level interconnects similar to modern memory. The memory hierarchy represents the internal memory partitioning and contains the hierarchy-related parameters such as data transfer bandwidths (*Bandwidth*) using different interconnect links.

4.1.2 Architecture-Specific Modeling. With the hardware model, we can represent different PIM architectures in OptiPIM by changing architecture-specific parameters. Figures 4(b) and (c) show the configurations for the near-bank processing of HBM-PIM [35] and the in-situ bit-serial processing in DRAM [16], respectively. In HBM-PIM [35], the data is stored in a horizontal layout (i.e., Figure 3(c)); therefore, *Row_Size* is the HBM row size divided by the bit precision. The computation throughput is set to the number of SIMD slots that fully exploit the bank-level data link. In SIMDRAM [16], computation requires a vertical layout (i.e., Figure 3(d)) so that each memory row can process *Row_Size* number of elements. We can configure *#Rows* as the number of DRAM rows divided by the bit precision. Furthermore, SIMDRAM conducts in-memory computation using multi-row activations, which can process a whole row

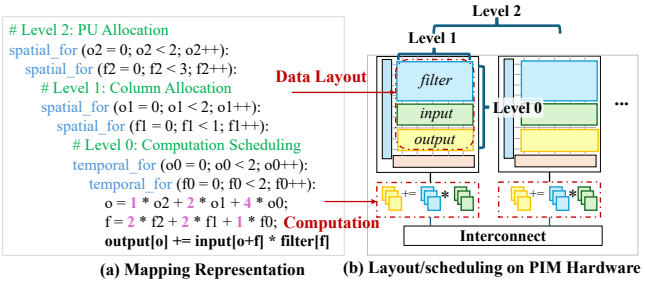


Figure 5: OptiPIM mapping representation on PIM hardware.

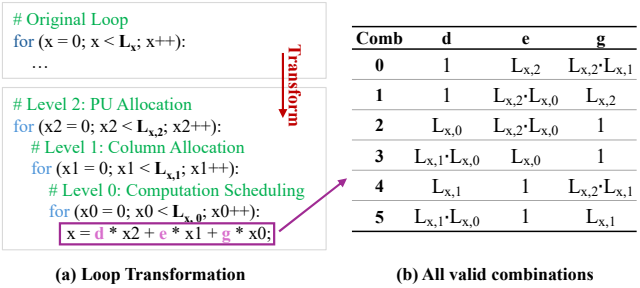
simultaneously. Therefore, the computation throughput of SIMDRAM is similar to *Row_Size*, while *#Rows* equals the number of actual memory rows divided by the bit precision. We can also configure the computation latency and energy consumption according to the bit-serial processing pattern, whose complexity increases quadratically with the bit precision. This general PIM model enables us to construct a framework for mapping on various PIM architectures.

4.1.3 Costs in General PIM Model. Figure 4(a) also shows different costs of PIM mapping in our model, including data layout, computing, data loading, and output gathering. Specifically, data layout is the arrangement of the specific rows and columns to accommodate the data for PIM computations. The requirements of data layout depend on the PIM technology. For example, HBM-PIM reserves the memory space for stationary weights (e.g., a weight matrix in a fully connected layer and a convolution filter) while storing inputs and outputs in registers. On the other hand, SIMDRAM must reserve memory space for all operands for in-memory computation. The data loading cost refers to loading data in different places, depending on the layout and architecture bandwidth. The computation cost is calculated based on the operation latency on a specific PIM architecture. The output gathering cost refers to the overhead incurred when transferring the computation results from the PIM unit to the host. The output size for gathering depends on the PIM technology. If the architecture supports in-memory multiply-and-accumulate (MAC) operations [35, 45], the host only gathers the final MAC result. If the architecture lacks the support for in-memory MAC [16], the host must read all partial products of element-wise multiplications.

4.2 PIM Layout-Aware Mapping Representation

To efficiently and effectively describe the design space of PIM mapping, we propose a layout-aware nested loop representation, as shown in Figure 5. The nested loop representation of OptiPIM enforces spatial and temporal levels in the loop structure to reflect the PIM-specific hardware characteristics, where a valid loop nest must have at least three continuous regions: PU allocation, column allocation, and computation scheduling. Separating the loop levels in this way is conceptually analogous to applying loop tiling and vectorization, where each level isolates a specific granularity of parallelism and data locality.

4.2.1 PU Allocation. PU is the basic hardware component in our PIM model. Therefore, the highest loop level determines the number



Comb	d	e	g
0	1	$L_{x,2}$	$L_{x,2}\cdot L_{x,1}$
1	1	$L_{x,2}\cdot L_{x,0}$	$L_{x,2}$
2	$L_{x,0}$	$L_{x,2}\cdot L_{x,0}$	1
3	$L_{x,1}\cdot L_{x,0}$	$L_{x,0}$	1
4	$L_{x,1}$	1	$L_{x,2}\cdot L_{x,1}$
5	$L_{x,1}\cdot L_{x,0}$	1	$L_{x,1}$

Figure 6: (a) The transformation conducted by OptiPIM. (b) All valid coefficient combinations for the indexing function of loop variable x.

of PUs used for computation in the PIM system and partial data indices distributed across different PUs (exact indices are determined by loop variables at all levels and the indexing function).

4.2.2 Column Allocation. In the middle level, the OptiPIM mapping representation explicitly indicates the computation allocation across different columns when processing a memory row in a PU and, thus, the detailed data layout in these columns. Ideally, when the number of memory columns in a PU is the same as the processing throughput (e.g., SIMDRAM), all columns can be executed in a single SIMD operation. When the number of columns is larger than the processing throughput (e.g., HBM-PIM), all columns in a PU are divided into SIMD lane-size tiles, requiring multiple sequential SIMD computations.

4.2.3 Computation Scheduling. The last loop level indicates the temporal computation scheduling in each column of a PU. It also enables us to directly generate the detailed data layout across different rows in each column specified by the data indexing function and the loop structure. In Section 5.5, we propose an algorithm that can accurately calculate the cost for storing data (e.g., input, weight, output) based on the mapping representation.

The ultimate target of OptiPIM is to transform a regular nested loop into the proposed representation that explicitly specifies how the operations of the loop will be mapped onto different components in the PIM architecture as shown in Figure 6.

4.3 Comprehensive Data Indexing

The nested loop represents the design space by specifying the loop bound for each loop variable at each loop level. Due to the layout constraints of PIM acceleration, the proposed nested loop can represent all the data "shapes" in the memory. However, the detailed distribution of data indices is determined by the indexing function based on specific values for each loop variable. Unfortunately, all prior mapping representations only consider one single data indexing function, which calculates the data index using tiled loop bound values from the innermost loop level to the outermost loop level (e.g., f = 2 ∗ f0 + 2 ∗ f1 + 1 ∗ f2 in Figure 5). Fixing the indexing function rules out a large design space of valid data layouts in PIM architectures. In OptiPIM, we consider a more comprehensive design space than prior methods by exploring transformations of the data indexing function.

As illustrated in Figure 6(a), the loop variable x is divided into three loop levels: $0 \le x_2 < L_{x,2}$, $0 \le x_1 < L_{x,1}$, and $0 \le x_0 < L_{x,0}$, where $L_{x,2} \cdot L_{x,1} \cdot L_{x,0} = L_x$ and L_x denotes the loop bound of x in the original nested loop. The data index is determined through a linear transformation $d * x_2 + e * x_1 + f * x_0$ using the transformation coefficients d, e, and g, where $d, e, g \in \mathbb{Z}^+$. To ensure a valid loop transformation, the loop semantics must remain consistent before and after the transformation. Consequently, the indexing must satisfy the following constraints:

- The linear transformation function of x must be **bijective**, as we have L_x combinations of (x_2, x_1, x_0) that have to be one-to-one mapped to L_x values of x.
- The range of the linear transformation must encompass the entire interval range $[0, L_x - 1]$.

We found that only **6 combinations** of the transformation coefficients d, e, and g are feasible when $L_{x,2} \ge 1$, $L_{x,1} \ge 1$, and $L_{x,0} \ge 1$ under the above constraints. These combinations are illustrated in Figure6(b). Building on this finding, we can incorporate the indexing function as an additional design dimension in the mapping optimization, thereby expanding the design space without significantly increasing its complexity. We briefly prove this finding as follows.

Proof. We define $\mathbb{F}(x_2, x_1, x_0)$ as the indexing function:

$$\mathbb{F}(x_2, x_1, x_0) = d \cdot x_2 + e \cdot x_1 + g \cdot x_0. \tag{1}$$

Given all the constraints: ① $\mathbb{F}(x_2, x_1, x_0)$ must be a bijective linear transformation from $\mathbb{R}^3$ to $\mathbb{R}$, ② $\mathbb{F}(L_{x,2} - 1, L_{x,1} - 1, L_{x,0} - 1) = L_{x,2} \cdot L_{x,1} \cdot L_{x,0} - 1$, and ③ One of the following: $\mathbb{F}(1, 0, 0) = d$, $\mathbb{F}(0, 1, 0) = e$, or $\mathbb{F}(0, 0, 1) = g$ must be equal to 1. This requirement guarantees that $\mathbb{F}(x_2, x_1, x_0)$ spans the entire range $[0, L_x - 1]$, with d, e, and g representing the smallest non-zero values achievable by the transformation.

We need to consider three sub-cases satisfying constraint ③. We begin by analyzing the possible coefficients when $d = 1$. In this scenario, Equation 1 becomes $\mathbb{F}(x_2, x_1, x_0) = x_2 + e \cdot x_1 + g \cdot x_0$. Moreover, $x_2 \in [0, L_{x,2} - 1]$ increases continuously without any gaps in the range. Then, the increment caused by $e \cdot x_1 + g \cdot x_0$ must equal $L_{x,2}$ to satisfy the bijectivity constraint (no overlaps). That means e and g can be represented as:

$$e = p \cdot L_{x,2},\ g = q \cdot L_{x,2}.$$

Where, $p > 0, p \in \mathbb{Z}^+$, and $q > 0, q \in \mathbb{Z}^+$. Therefore, $\mathbb{F}(x_2, x_1, x_0)$ can be rewritten as:

$$\mathbb{F}(x_2, x_1, x_0) = x_2 + L_{x,2} \cdot (p * x_1 + q * x_0).$$

Based on constraint ②, we have:

$$p \cdot (L_{x,1} - 1) + q \cdot (L_{x,0} - 1) = L_{x,1} \cdot L_{x,0} - 1. \tag{2}$$

We notice that Equation 2 is a linear diophantine equation (LDE) [6], since $L_{x,1} \cdot L_{x,0} - 1 = L_{x,1} \cdot (L_{x,0} - 1) + (L_{x,1} - 1)$, which means $gcd(L_{x,1}-1, L_{x,0}-1)$ divides $L_{x,1} \cdot L_{x,0} - 1$, where $gcd(L_{x,1}-1, L_{x,0}-1)$ is the greatest common divisor of $L_{x,1} - 1$ and $L_{x,0} - 1$. Thus, there exist integer solutions for Equation 2. Moreover, based on the bijectivity constraint, $p \cdot x_1 + q \cdot x_0$ must also be bijective and cover the entire range of $[0, L_{x,1} \cdot L_{x,0} - 1]$. Thus, one of p and q must be 1. Then, based on Equation 2, p and q can only take the following

Table 1: ILP Variable and Notation Declarations

Output Variables	
$L_{x,l}$	The loop bound of x at level l.
$C_{x,l}$	The value of the l_{th} coefficient of loop variable x.
Internal Variables	
$\beta^k_{x,l}$	Binary variable indicates whether the k_{th} divisor of L_x is selected at loop level l.
$\delta_{x,f}$	Binary variable indicates whether the f_{th} coefficient combination is selected for loop variable x.
$c^l_{x,f}$	The value of the l_{th} transformation coefficient in the f_{th} combination of loop variable x.
Application Constants	
L_x	Original loop bound value of loop variable x.
$\mathbf{D_x}$	Vector comprising all divisors of L_x.
$\mathbf{U}$	Set of all loop variables.
$\mathbf{S_v}$	Set of loop variable index related to tensor v.
Architecture Constants	
$\mathcal{BW}_c$	Bandwidth for a hardware component c.
$\mathcal{P}$	Data precision for the data elements.
$\mathcal{TP}$	SIMD computation throughput in each PU.
$\mathbf{N_c}$	Number of components (e.g., column, row, bank, etc.)
Notations	
k	Divisor index for a specific loop bound.
x	Loop variables in the original loop.
l	Loop level index in the transformed nested loop.
f	Coefficients combination index of a loop variable.

two pairs of values:

$$\begin{cases} p_0 = 1, q_0 = L_{x,1} \\ p_1 = L_{x,0}, q_1 = 1 \end{cases} \tag{3}$$

Consequently, e and g are restricted to two pairs of values shown in the first two rows of Figure 6(b). Similar proofs hold for the cases where $e = 1$ or $g = 1$. Therefore, in the scenario involving *three loop levels*, the coefficients are limited to $3 \times 2 = 6$ possible combinations.

Our findings can be **generalized to any number of loop levels**. Assume we have l loop levels after transformation, the number of possible combinations of coefficients ($\text{Num}_{\text{coeff}}$) in the indexing function of a single loop variable is:

$$\text{Num}_{\text{coeff}} = l! \tag{4}$$

With this comprehensive yet scalable indexing method, we can enumerate every valid data layout for a given PIM architecture; in our target case of three loop levels, this requires only six coefficient combinations (see Figure 6).

5 Problem Formulation and Optimization

In this section, we formulate an ILP model that, given an unmapped and unoptimized nested-loop workload, produces the optimal PIM mapping expressed in our proposed representation (see Figure 6). We detail all variables, constants, and notations of our ILP in Table 1.

5.1 Variables for Nested Loop Representation

Before delving into the cost functions and constraints, we first define variables for loop bounds and data indexing in accordance with the representation introduced in Section 4.

5.1.1 Variables for Loop Bounds. For each original loop variable x (e.g., o or f in 1D convolution in Figure 2), we assume a perfect factorization of the original loop bound L_x to establish loop bounds ($L_{x,l}$) across different tiled loop levels. Thus, all $L_{x,l}$ can only be the divisors of L_x. We define $\mathbf{D_x}$ as the vector of all divisors of L_x in ascending order [52]. For example, if $L_x = 8$, then, $\mathbf{D_x} = [1, 2, 4, 8]^T$. We use a one-hot encoding vector $\mathbf{B_{x,l}}$, defined as:

$$\mathbf{B_{x,l}} = [\beta^0_{x,l}, \ \dots, \ \beta^k_{x,l}, \ \dots], \tag{5}$$

to indicate which divisor in $\mathbf{D_x}$ is selected to be the transformed loop bound of x at level l. Thus, the value of $L_{x,l}$ can be calculated as $\mathbf{B_{x,l}} \cdot \mathbf{D_x}$. For example, if $L_{x,2} = 4$, it can be expressed as:

$$L_{x,2} = \mathbf{B_{x,2}} \cdot \mathbf{D_x} = [0, \ 0, \ 1, \ 0] \cdot [1, 2, 4, 8]^T = 4. \tag{6}$$

5.1.2 Variables for Data Indexing Coefficients. As illustrated in Section 4.3 and Figure 6(b), for each original loop variable x, there are **6** possible combinations of linear transformation coefficients for data indexing, after transforming it into **3 layout-aware loop levels**. Each combination can be represented using the loop bound variables. For example, the first combination (comb 0 in Figure 6(b)) of coefficients can be represented as $c^0_{x,0} = 1$, $c^1_{x,0} = L_{x,2}$ and $c^2_{x,0} = L_{x,2} \cdot L_{x,1}$.

We define binary variables $\delta_{x,f}$ to indicate which combination is selected. Thus, the value of the l_{th} transformation coefficient of loop variable x can be calculated as:

$$C_{x,l} = \textstyle\sum_{f=0}^{5} c^l_{x,f} \cdot \delta_{x,f}. \tag{7}$$

5.2 Constraints for Nested Loop Representation

To ensure a valid nested loop representation for the given operator, we need the following constraints in the ILP formulation.

5.2.1 Loop Bound Constraints. Since we use one-hot encoding vectors to select the loop bound values, only one divisor can be selected for loop variable $\mathbf{x}$ at level $\mathbf{l}$:

$$\textstyle\sum_{k \in [0, \ \text{len}(\mathbf{D_x})-1]} \beta^k_{x,l} = 1, \forall x, l \tag{8}$$

where, $\text{len}(\mathbf{D_x})$ is the length of $\mathbf{D_x}$. For example, in Equation 6, only one value can be 1 in $\mathbf{B_{x,2}}$, which is $\beta^2_{x,2}$.

Besides that, we need to preserve the value of the original loop bound L_x. For example, in Figure 6, we have $L_{x,2} \cdot L_{x,1} \cdot L_{x,0} = L_x$. We have the following constraint:

$$\textstyle\prod_{l=0}^{2} \mathbf{B_{x,l}} \cdot \mathbf{D_x} = L_x, \forall x. \tag{9}$$

5.2.2 Data Indexing Coefficients Constraint. For each loop variable x, only one of the six coefficient combinations can be selected:

$$\textstyle\sum_{f=0}^{5} \delta_{x,f} = 1, \forall x. \tag{10}$$

5.3 Variables for Data Layout

OptiPIM's representation explicitly determines the data layout, and we directly compute all data layout related information from the loop-bound variables in the ILP formulation.

5.3.1 Number of Required PUs. We use Num^{sys}_{PU} to denote the number of PUs required for a nested loop, calculated by the product of all level-2 loop bounds: $Num^{sys}_{PU} = \prod_{x \in \mathbf{U}} L_{x,2}$.

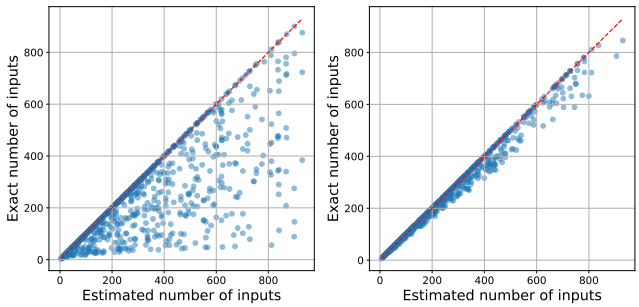


Figure 7: The estimation error of Num_{input}^{col} from (a) COSA's method [23], and (b) the proposed method. OptiPIM's estimation method is more accurate than COSA's, with an error of only 3 percent on average.

5.3.2 Number of Columns in a PU. We use Num_{col}^{PU} to denote the number of columns required for a nested loop in a PU. Due to the data layout enforcement illustrated in Section 3.1, we cannot reuse any data across different columns for PIM operations. Therefore, we calculate the required columns by the product of all level-1 loop bounds: $Num_{col}^{PU} = \prod_{x \in \mathbf{U}} L_{x,1}$.

5.3.3 Number of Rows in a Column. Unlike Num_{col}^{PU}, we need to calculate the number of rows for each tensor v. Therefore, we use Num_{v}^{col} to denote the number of rows required for a tensor v in each PU column for a nested loop. Num_{v}^{col} is also equal to the number of unique indices of tensor v required by all computations in a column. For the indexing schemes independently specified by the loop variables (e.g., output and filter in convolution), the number of required rows can be calculated by: $Num_{v}^{col} = \prod_{x \in \mathbf{S}_v} L_{x,0}$, where $\mathbf{S}_v$ denotes the set of loop variables that index the tensor v. In Figure 5, we have $\mathbf{S}_{\text{outs}} = \{o\}$. However, when the indexing scheme is determined by a linear combination of loop variables, calculating Num_{v}^{col} becomes more complex. In Figure 5, a linear combination of o and f defines the input indexing scheme. Therefore, Num_{input}^{col} corresponds to the number of unique input indices generated by the expression $a \cdot o_0 + b \cdot f_0$, where $a = 4$ and $b = 1$ in this example.

Prior ASIC-based frameworks (e.g., COSA [23]) directly multiply all related loop bounds to calculate Num_{input}^{col}, leading to significant overestimation. Figure 7(a) illustrates the mismatch between actual input elements (y-axis) and COSA's estimates (x-axis) using 1000 random samples for $a, b, L_{o,0}$, and $L_{f,0}$, where the red line shows the perfect estimation. We observe that COSA's method introduces over 70% errors for most sampling points.

In this work, we propose the following estimation function for the calculation of unique inputs:

$$Num_{input}^{col} = \min\{\frac{a \cdot (L_{o,0} - 1) + b \cdot (L_{f,0} - 1)}{gcd(a, b)} + 1, L_{o,0} \cdot L_{f,0}\} \quad (11)$$

The first term of Equation 11 represents the number of unique values produced by the linear function when the ranges of $L_{o,0}$ and $L_{f,0}$ are similar [49, 51]. The second term indicates the maximum number of unique values possible in the nested loop. The results of our proposed estimation function are shown in Figure7(b), demonstrating only an average 3.2% error from the ideal estimation. All equations can be linearized and encoded into the ILP model using the same linearization method as prior works [23, 52], along with the capability of optimizers like Gurobi [15] to handle non-linear constraints.

5.4 Constraints for Data Layout

To generate a valid data layout for a specified architecture, the resulting loops must satisfy the following architectural constraints.

5.4.1 Number of PUs Constraint. Num_{PU}^{sys} cannot exceed the number of PUs ($\mathbf{N_{PU}}$) allocated for this operator:

$$Num_{PU}^{sys} \leq \mathbf{N_{PU}}. \quad (12)$$

5.4.2 Row Size Constraint. Num_{col}^{PU} cannot exceed the maximum number of columns in a single PU ($\mathbf{N_{cols}}$):

$$Num_{col}^{PU} \leq \mathbf{N_{cols}}. \quad (13)$$

5.4.3 Column Size Constraint. The total size of tensor elements stored in a single column can not exceed the maximum capacity of a column within a PU (i.e., $\mathbf{N_{row}}$). As illustrated in Section 4.1.3, the tensors that need to be stored in the memory depend on the PIM technology and the operator type. Therefore, the column size constraint is as follows:

$$\sum_{v \in \mathbf{S}_{\text{mem}}} Num_{v}^{col} \leq \mathbf{N_{row}} \quad (14)$$

where $\mathbf{S}_{\text{mem}}$ denotes tensors stored in memory. For example, in a fully connected operator, $\mathbf{S}_{\text{mem}} = \{\text{weight}\}$ on HBM-PIM and $\mathbf{S}_{\text{mem}} = \{\text{in}, \text{weight}\}$ on SIMDRAM.

5.5 PIM Cost Modeling

In this section, we employ loop variables, combined with application-specific and PIM architecture-specific information, to model the performance cost of a mapping on PIM accelerators.

5.5.1 Computation Cost. Recent PIM accelerators support parallel computations across multiple PUs (e.g., all-bank execution in HBM-PIM [35]). Therefore, the computation cost depends on computations in each PU, specified by the level-0 and level-1 loop variables. Specifically, level-0 loop variables indicate the sequential computations in a column, and level-1 loop variables indicate the computations placed in a memory row.

The sequential computations in a column include two operations: partial product multiplications and reductions. We can calculate the number of multiplications in a column based on all loop-bound variables at level 0:

$$Num_{mul}^{col} = \prod_{x \in \mathbf{U}} L_{x,0}. \quad (15)$$

The number of reductions in a column can be calculated as:

$$Num_{red}^{col} = (\prod_{x \in \hat{\mathbf{S}}_{\text{outs}}} L_{x,0} - 1) \cdot Num_{out}^{col}. \quad (16)$$

Here, $\hat{\mathbf{S}}_{\text{outs}} = \mathbf{U} - \mathbf{S}_{\text{outs}}$ is the complement set of $\mathbf{S}_{\text{outs}}$ in the set of all loop variables, as iterating over variables in $\hat{\mathbf{S}}_{\text{outs}}$ results in the same index in the output tensor.

The costs of computations in a memory row, $Cost_{row}^{mul}$ and $Cost_{row}^{add}$, depend on the PIM technology. For example, in SIMDRAM [16],

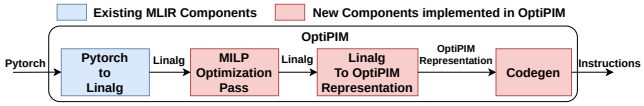


Figure 8: The flow of OptiPIM.

$Cost^{mul}_{row}$ and $Cost^{add}_{row}$ are the costs of required bit-serial AAP operations. In HBM-PIM [35], both $Cost^{mul}_{row}$ and $Cost^{add}_{row}$ include row activation for data stored in the memory, sequential memory reads from the activated row to registers, and sequential computations in NBU based on $\mathcal{TP}$. We can calculate the computation cost as:

$$Cost_{\text{compute}} = Cost^{mul}_{row} \cdot Num^{col}_{mul} + Cost^{add}_{row} \cdot Num^{col}_{red}. \quad (17)$$

5.5.2 Output Gathering Cost. As outlined in Section 4, the host gathers output results after PIM computations. The output gathering cost can be calculated as:

$$Cost_{\text{output}} = \frac{\sum_{PU \in sys} Size^{PU}_{out}}{\mathcal{BW}_c}. \quad (18)$$

Here, $\sum_{PU \in sys} Size^{PU}_{out}$ is the size of all output-related elements (partial products and final results), and $\mathcal{BW}_c$ is the memory bandwidth to transfer these elements. $Size^{PU}_{out}$ depends on the output data layout and PIM technology. For example, SIMDRAM [16] does not support reduction in a row, requiring the host to gather all partial products from each PU. HBM-PIM [35] supports MAC operation in NBU, so the host only gathers one partial product for each output from a PU. $\mathcal{BW}_c$ depends on the memory hierarchy.

5.5.3 Input Loading Cost. Input loading cost indicates the loading of the non-stationary data to PUs; it depends on layout and PIM technology:

$$Cost_{\text{input}} = \frac{\sum_{PU \in sys} Num^{PU}_{input} \cdot \mathcal{P}}{\mathcal{BW}_c}. \quad (19)$$

Here, $\mathcal{P}$ denotes the data precision.

5.5.4 Objective Function. The objective function for a workload is the sum of all the costs:

$$\text{Obj} = Cost_{\text{compute}} + Cost_{\text{output}} + Cost_{\text{input}}. \quad (20)$$

ILP minimizes Obj to enhance the overall performance.

5.6 Multi-Operator Optimization

PIM accelerators can accommodate multiple operators—each expressed as a single nested loop—concurrently due to the large memory capacity. Our ILP model can be extended to optimize the execution of multiple operators and automatically determine the appropriate number of PUs allocated to each operator in the target workload. If $\mathbf{N_{PU}}$ is assigned to a set of operators ($\mathcal{W}$), we can modify Equation 12 as follows:

$$\sum_{i \in \mathcal{W}} Num^{i}_{PU} \leq \mathbf{N_{PU}}.$$

Here, Num^{i}_{PU} is the number of PUs used by operator i, and the ILP objective is to minimize $\sum_{i \in \mathcal{W}} \text{Obj}_i$, thereby improving the performance across all operators in $\mathcal{W}$ collectively.

Table 2: Architectural parameters.

HBM configuration	8-high HBM2e (8Gb/channel)
Memory organization	#banks/channel=32, #channels/stack=16
Bank specification	32MB, row_size=1kB
Data transfer	DQ size = 256-bit
Timing (cycles)	tRC : 19, $tRAS$: 17, tRP : 7, $tFAW$: 15

6 Experimental Setup

6.1 Compiler and Optimization Implementation

End-to-end flow and abstractions: Figure 8 illustrates OptiPIM's flow, which we integrate into MLIR [34]. We lower Pytorch workloads using Torch-MLIR [39] to the Linalg dialect, which serves as input to the ILP optimization pass. The ILP model is implemented with Gurobi [15] (v11.03), which generates the optimized mapping described by our representation. OptiPIM then performs recursive loop induction variable analysis on a compact format of this representation (see Figure 12) to infer the temporal and spatial data spaces based on the determined loop bounds during architecture-specific code generation (codegen) for HBM-PIM and SIMDRAM, together with the information of types of operations in the workload (e.g, MACs). The codegen process allocates memory and buffers for data and generates hardware instructions. Based on our experiments, the maximum optimization time for one machine learning operator is less than 4 minutes on AMD Ryzen 7 Pro 7840U (3.2GHz) and 32GB RAM.

Buffer allocation and codegen: SIMDRAM codegen allocates memory rows for data and computation, following the restriction of bit-serial in-situ computation (e.g., reserved rows dedicated for buffering operands and results for computation) [16]. Then, SIMDRAM codegen generates corresponding activate-activate-precharge (AAP) operations for bit-serial row-parallel computations. The HBM-PIM codegen allocates memory space for weights and registers for inputs and outputs. The register allocation strategy allocates registers for chunks of vector operations in the order specified by the nested loop. Specifically, we allocate 8 general registers (each has 16×16-bit) for inputs and 8 general registers for outputs, the same as the strategy used in HBM-PIM [54]. Both codegens generate memory read/write operations for data communications via the host. The end-to-end framework is open-sourced (see A.2).

6.2 Hardware Configuration

We use HBM2E [44] as the basic architecture for both HBM-PIM (2 banks per NBU) and SIMDRAM, as shown in Table 2. The memory is connected to a host at 500GB/s bandwidth [44]. To evaluate the performance of the generated mappings, we use an analytical model that has been cross-verified with cycle-accurate simulators [41] for HBM-PIM [35] and SIMDRAM [16]. This analytical model allows us to efficiently assess our mappings on different PIM architectures. We assume HBM-PIM uses a 16-bit floating-point data type, and SIMDRAM computes with 16-bit fixed-point data.

6.3 Workloads

We evaluate OptiPIM on several widely used convolutional and transformer neural networks, including AlexNet [29], ResNet-50 [17],

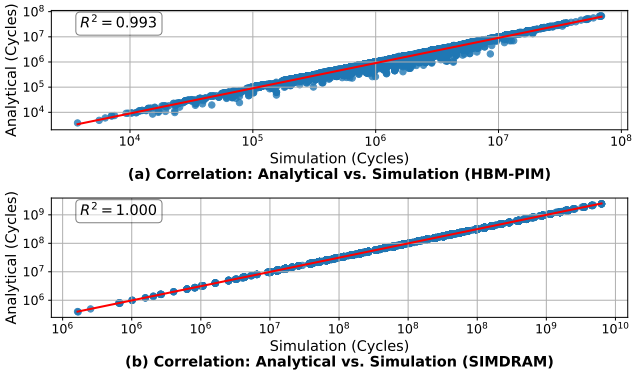


Figure 9: Validation of analytical model against Ramulator 2 [41] with PIM extension.

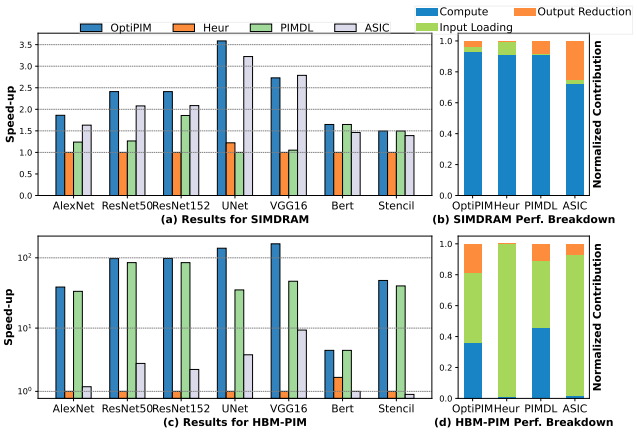


Figure 10: Overall performance results and breakdown of different mapping algorithms for both PIM architectures.

ResNet-152 [17], UNet [22], VGG [55], BERT-base [10], and Llama-3 8B and 14B models [14, 50]. We also tested another nested loop workload, Stencil [48]. This covers a wide range of data-intensive operators with varying configurations. All workloads are run under 16-bit data precision.

6.4 Baseline Mappings

6.4.1 Weight-Parallel Heuristic [35]. We implement a baseline heuristic based on the mapping strategy in HBM-PIM [35], which divides the weight matrix into multiple tiles, where the tile size is set to match the number of SIMD lanes. The HBM-PIM heuristic distributes these weight matrix tiles in the order of bank, column, and row to fully exploit the parallelism of all-bank execution. We note that the original HBM-PIM heuristic only supports matrix-vector multiplication. For more complicated kernels like convolution, we distribute the convolution filters in a similar way across different banks and load the corresponding input in the SIMD computation.

6.4.2 Search-Based Method [7, 65]. Existing PIM frameworks [7, 65] rely on exhaustive/random search to find efficient mappings. We implement the method used by PIM-DL [65] based on a genetic algorithm. We cap the genetic algorithm runtime at 1 hour for each operator. We also implement a full exhaustive search to validate OptiPIM's optimality (Section 7.5.3).

6.4.3 ASIC-Based Mapping Method [23]. Prior ASIC-based mapping techniques did not consider PIM-specific costs, constraints, and design spaces. We adopt COSA's ILP optimization [23] on our PIM accelerators without considering the fine-grained data layout-related costs in the objective function to reflect the necessity of PIM-specific problem formulation.

7 Evaluation

7.1 Validation of Analytical Model

We cross-verify our proposed analytical cost models with the cycle-accurate simulator (Ramulator 2 [41]), which is extended to support various PIM instructions of HBM-PIM [35] and SIMDRAM [16]. We randomly generate 12000 valid mappings from all our benchmarks, and obtain the performance results from both our analytical models and the simulator. Figure 9 shows our analytical models accurately align with the cycle-accurate simulation with the squared Pearson correlation coefficient (R^2) greater than 0.99 for both architectures.

7.2 Comparison to Existing Frameworks

7.2.1 SIMDRAM [16]. Figure 10 (a) shows the overall performance of the tested workloads on SIMDRAM [23] using different mapping methods. Figure 10(b) shows the detailed cost breakdown of each method, indicating that in-memory computation dominates the execution time on SIMDRAM [16] because of bit-serial processing. Compared to the heuristic baseline and the PIM-DL's random search method, OptiPIM can constantly find more efficient mappings, yielding 2.3× and 2.0× speedup (averaged across all benchmarks), respectively. Compared to the ASIC mapping algorithm, OptiPIM generates mappings that are 1.1× faster on average. The less significant speedup than in the heuristic and search baselines is due to the fact that the neglected layout-related costs (e.g., input loading and output reduction) are not dominant in SIMDRAM execution, as suggested by Figure 10(b). However, our experiments show that, even in compute-dominated frameworks, considering layout-related costs is necessary for finding optimal mappings.

7.2.2 HBM-PIM [35]. Figure 10(c) and (d) show the overall performance and cost breakdown of the tested workloads on HBM-PIM [35] using different mapping methods, showing distinct results from experiments on SIMDRAM [16]. Specifically, our experiments show that OptiPIM is 72.8×, 1.8×, and 25.4× better than HBM-PIM's heuristic, PIM-DL, and ASIC, respectively. The reason why HBM-PIM's heuristic cannot provide satisfactory performance is that the heuristic is originally designed for matrix-vector multiplication, which can fully parallelize the computation for partial products by distributing the weight matrix while reusing the input vector in the registers. For BERT, which mainly consists of matrix-vector multiplication operators, HBM-PIM's heuristic is only 2.4× slower than OptiPIM. However, when applying this heuristic to convolution, it becomes hard to reuse the input vector because of the data misalignment, forcing the data loading for different input indices. Even though the HBM-PIM's heuristic has a lower computation

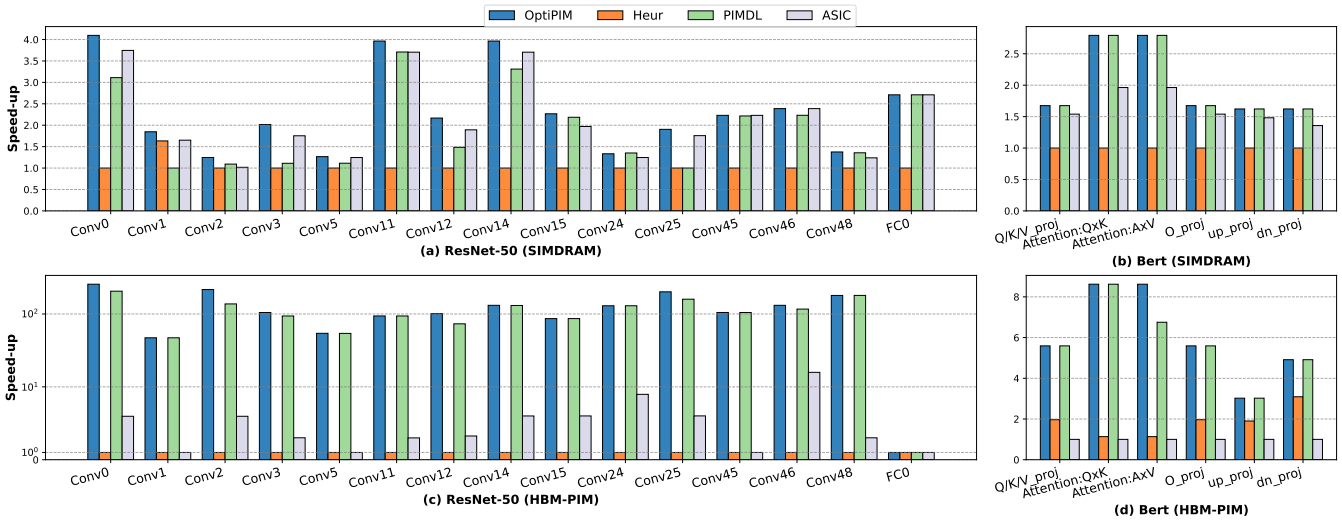


Figure 11: Operator breakdown results.

```
# ResNet50 – Layer Conv12
# N: Batch
# K, P, Q: Out. Channel, Height, Width
# C: In. Channel
# R, S: Filter Height, Width
Loop Variables: N, K, P, Q, C, R, S
Loop Bounds: 1, 128, 28, 28, 128, 3, 3
Stride: 2, 2
Dilation: 1, 1
```

(a) Original operator information

```
# Our – HBM-PIM
Loop Variables: N, K, P, Q, C, R, S
Loop Bounds:
  Level 2: 1, 1, 1, 1, 16, 1, 1;
  Level 1: 1, 2, 2, 4, 8, 1, 1;
  Level 0: 1, 64, 14, 7, 1, 3, 3;
Coefficients:
  K2, K1, K0: [128, 64, 1]; P2, P1, P0: [28, 14, 1];
  Q2, Q1, Q0: [28, 7, 1]; C2, C1, C0: [8, 1, 1];
Costs:
  Input Loading: 18432; Computation: 13248;
  Output Reduction: 6272;
```

(b) OptiPIM result

```
# Heuristic – HBM-PIM
Loop Variables: N, K, P, Q, C, R, S
Loop Bounds:
  Level 2: 1, 32, 1, 1, 1, 1, 1;
  Level 1: 1, 4, 28, 4, 1, 1, 1;
  Level 0: 1, 1, 1, 7, 128, 3, 3;
Coefficients:
  K2, K1, K0: [4, 1, 1]; P2, P1, P0: [28, 1, 1];
  Q2, Q1, Q0: [28, 7, 1]; C2, C1, C0: [128, 128, 1];
Costs:
  Input Loading: 3784704; Computation: 49536;
  Output Reduction: 14;
```

(c) Heuristic result

```
# ASIC – HBM-PIM
Loop Variables: N, K, P, Q, C, R, S
Loop Bounds:
  Level 2: 1, 1, 2, 1, 16, 1, 1;
  Level 1: 1, 32, 1, 1, 4, 1, 3;
  Level 0: 1, 4, 14, 28, 2, 3, 1;
Coefficients:
  K2, K1, K0: [128, 4, 1]; P2, P1, P0: [14, 14, 1];
  Q2, Q1, Q0: [28, 28, 1]; C2, C1, C0: [8, 2, 1];
Costs:
  Input Loading: 1806336; Computation: 936;
  Output Reduction: 3136;
```

(d) ASIC result

Figure 12: Generated loops for ResNet50 Conv12.

and output reduction cost than OptiPIM, it introduces an extremely high input loading cost. The lack of consideration of input loading cost also results in the significant slowdown of ASIC (Figure 10(d)).

7.3 Operator Breakdown Results

Figure 11 shows operator-wise mapping results generated by different methods for representative operators in ResNet-50 and BERT. The results show that OptiPIM consistently generates superior mappings across all operator types.

7.3.1 SIMDRAM. As analyzed in Section 7.2, SIMDRAM execution is dominated by in-memory computation. Therefore, both OptiPIM and ASIC-based mapping methods can generate good mappings for most convolution operators that are compute-intensive, outperforming both heuristic and PIM-DL baselines. However, the ASIC method performs worse than the guided-search method on BERT's operators, which are data-intensive matrix-matrix multiplications, especially attention (i.e., QK and AV).

7.3.2 HBM-PIM. On HBM-PIM, OptiPIM outperforms the ASIC-based method baseline more significantly than in SIMDRAM because the costs for aligning data in SIMD registers dominate the execution time in HBM-PIM. We also observe that PIM-DL's random search is effective for most operators in ResNet-50 and BERT. However, the random search method results in a large variance in the performance of the resulting mappings and also requires a much longer run time than OptiPIM (Section 6).

7.4 Detailed Mapping Analysis

Figure 12 shows the detailed mappings generated by different methods for the Conv12 operator in ResNet-50. We observe different cost (performance) breakdowns in various methods. Specifically, the original HBM-PIM heuristic distributes the filters across the spatial elements (banks and DRAM columns) in the order of N, K, R, and S, without interleaving. This heuristic results in a low computation cost and output reduction cost because the heuristic fully parallelizes the computations while keeping each output value in the same bank. However, the generated mapping results in an extremely high cost of input loading due to the layout enforcement of SIMD processing. The ASIC-based method can find the lowest computation cost in the design space. However, neglecting the fine-grained layout costs leads to high input loading time, significantly offseting the savings on computation. OptiPIM can find the best mapping by comprehensively considering costs in PIM acceleration.

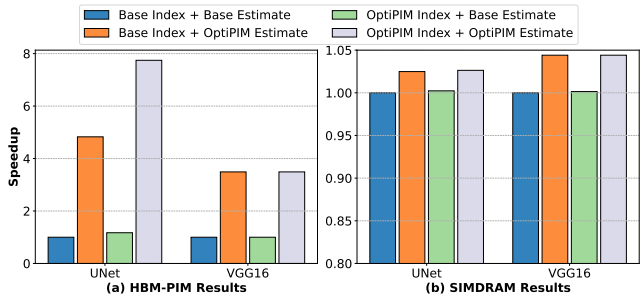


Figure 13: Optimization comparison.

7.5 Ablation Study

We evaluate algorithms that adopt different combinations of indexing and cost estimation methods. For the indexing method, we compare the proposed comprehensive indexing, *OptiPIM Index* (Section 4.3), to the baseline indexing *Base Index* that uses stride-1 indexing. Furthermore, we compare the proposed accurate cost estimation, *OptiPIM Estimate* (Section 5.3), to the product-based estimation used by COSA [23], denoted as *Base Estimate*. Figure 13 shows the performance of four different combinations on representative workloads, UNet, and VGG16.

7.5.1 Effect of Accurate Estimation. The accurate estimation of data layout-related costs can provide 4.8× (3.5×) and 6.6× (3.5×) improvements with *Base Index* and *OptiPIM Index* on UNet (VGG-16) on HBM-PIM. However, the accurate estimation provides less improvements on SIMDRAM, which are 2.5% (4.4%) and 2.4% (4.3%) with *Base Index* and *OptiPIM Index* on UNet (VGG16), because layout-related costs (e.g., input loading and output reduction) are significantly smaller than in-memory computation costs.

7.5.2 Effect of Comprehensive Indexing. The comprehensive indexing shows improvements on UNet: 1.2× speedup with *Base Estimate* and 1.6× speedup with *OptiPIM Estimate* on HBM-PIM. However, the comprehensive indexing provides trivial improvements on VGG16. In our model, comprehensive indexing primarily affects input loading. As shown in Equation 11, when the greatest common divisor (gcd) of the involved loop variables is greater than 1, it allows for optimized input sizes through strategic selection of coefficient combinations. In the case of UNet, multiple operators exhibit gcd values greater than 1, enabling the indexing functions to significantly reduce input loading sizes. Conversely, most operators in VGG16 have a gcd value equal to 1, which limits the effectiveness of comprehensive indexing for this workload.

7.5.3 Optimality of OptiPIM. Our results show that OptiPIM can find the optimal results on all tested workloads in 1 to 209 seconds for each operator on a consumer-grade 16-core CPU. We evaluate the optimality of OptiPIM by comparing it to the exhaustive search results obtained from parallelizing the search across 70 cores on an AMD EPYC 7713 server CPU (3.6GHz). As a comparison, the exhaustive search requires 9800 seconds (2.7 hours) on average to evaluate all possible mappings for each operator.

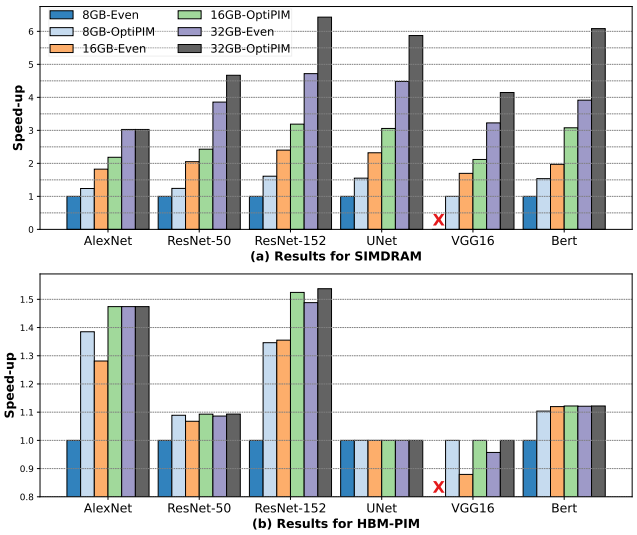


Figure 14: Comparison between multi-operator optimization and heuristic on systems with different memory capacities.

7.6 Scalability

7.6.1 Memory Scalability. We evaluate the scalability of OptiPIM, by running the same workloads on PIM architectures with different memory capacities. Figure 14 shows the performance of OptiPIM and a heuristic that evenly divides the memory resources for all operators in the workload. We scale the memory capacity from 8GB to 32GB to show the adaptiveness of OptiPIM on various workload-hardware scenarios. Based on our experiments, the OptiPIM algorithm is 43.6% (18.5%), 30.6% (7.3%), and 28.7% (1.4%) faster than the even distribution baseline on SIMDRAM (HBM-PIM) with 8GB, 16GB, and 32GB, respectively. The OptiPIM algorithm is more effective on SIMDRAM because the high-performance mappings on SIMDRAM require a large memory footprint, making resource allocation critical. On the contrary, HBM-PIM prefers the mappings with a low memory footprint, making 16GB sufficient to find the (near-)optimal mappings. When using 8GB memory, OptiPIM optimization becomes effective on HBM-PIM. The results show that OptiPIM optimization is important for large workloads that require more careful memory allocation for a large number of operators.

7.6.2 Model Scalability. When the model cannot fit into the PIM accelerator, OptiPIM divides the model into operator groups that can fit and executes them sequentially. As the memory needs to be updated before the start of each group, we add the corresponding weight loading costs to OptiPIM's objective function. To illustrate this approach, we conduct experiments on two large language models (LLMs) with different input token numbers on a 128GB 8-stack HBM-PIM system, which has a comparable memory capacity to an off-the-shelf GPU architecture (e.g., H200). We focus on the prefill phase, as OptiPIM assumes the shapes of the matrices are fixed; we discuss other workload types in Section 7.7. The results are shown in Figure 15: smaller LLMs (i.e., Llama-3 8B) fit entirely onto the device, whereas the larger ones are partitioned into operator groups and still successfully accelerated by OptiPIM.

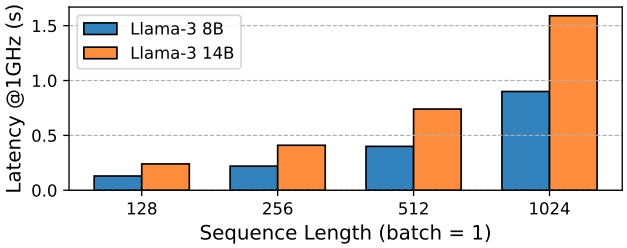


Figure 15: Evaluation of LLM prefill stage on HBM-PIM.

7.7 Discussion on the Scope of OptiPIM

OptiPIM is a compile-time optimization framework that generates optimal mappings for nested affine-loop operators with uniform computation patterns on digital PIM architectures. The nested loop format effectively represents many tensor programs such as machine learning, image processing, fluid dynamics, PDE solvers, and other high-performance computing tasks. OptiPIM supports mapping generation for both single-operator (ILP modeling) and multi-operator (Section 5.6) workloads. As a static optimization paradigm, it requires compile-time static shape information, and thus, dynamic-shape optimizations are beyond its current scope. To support dynamic shapes such as those in the LLM generation phase, OptiPIM can adopt similar approaches as those in current mainstream machine learning compilers by pre-optimizing for select shapes and padding operands at runtime [5, 43, 53]. Additionally, mainstream machine learning compilers such as TVM, TensorRT, and XLA [5, 43, 53] leverage a predefined library of architecture-specific operator (kernel) mappings and focus on higher-level transformations such as graph rewrite and kernel fusion. While such library-based approaches work well with conventional architectures such as GPU and TPU, they cannot capture the design space of PIM architectures. In contrast, OptiPIM swiftly navigates the design space to reach the optimal mapping of individual operators to PIM architectures.

8 Related Work

DNN Mapping Optimization. Several works [5, 19, 21, 31, 42, 47, 63, 64] utilize nested loops to model the spatial and temporal scheduling of computations for a single DNN operator on ASIC accelerators, exploring different loop orders and tiling strategies. COSA [23] and LEMON [52] expedite the search process by formulating Timeloop's [47] analytical model into an ILP model. Gamma [27] and Odyssey [61] apply genetic algorithms to identify optimized mappings. Additionally, Digamma [28] and DOSA [20] co-optimize both the accelerator's hardware configuration and the operator mapping. Other approaches [8, 58–60] introduce alternative representations, such as data-centric [4, 30] and relation-centric [40] models, to more effectively characterize the hardware and computational features of accelerators. As illustrated in Section 3, these optimization methods cannot effectively and efficiently optimize the mapping problem on PIM architecture.

Mapping in PIM Accelerators. Most prior accelerators [2, 11, 12, 25, 32, 35, 36, 38] adopt heuristics for operator mapping. For example, NeuralCache [11], DRISA [38], and ComputeDRAM [12] fully parallelize output elements of convolution and matrix multiplication onto different memory columns. Besides that, several prior works [7, 57, 62, 65] propose optimization frameworks to find optimized mappings on PIM accelerators. PIM-DL [65] proposes a framework to optimize the mapping of the whole neural network onto the logic-based PIM accelerators. ARES [7] proposes a general representation for mapping operators onto different PIM architectures. However, both PIM-DL [65] and ARES [7] explore the design space based on genetic algorithm and (guided) exhaustive search. SpecPIM [37] proposes a whole network mapping and scheduling algorithm for speculative inference in large language models (LLMs). However, it only considers operator types in LLMs and adopts heuristics without exact optimization. CimLoop [1] is a recent exhaustive-search mapping framework for analog-based PIM accelerators, which only considers the mapping problem as general matrix multiplication onto the crossbar-based analog-PIM accelerators. Unlike CimLoop [1], OptiPIM focuses on digital-based PIM accelerators that support the in-memory vector processing paradigm in conventional memory architecture, resulting in a significantly different design space than analog-based PIM accelerators.

9 Conclusion

In this work, we propose a novel optimization framework, OptiPIM, that systematically analyzes and addresses the mapping problem on PIM accelerators using ILP. OptiPIM utilizes a layout-aware representation and accurate cost models, enabling precise and efficient optimization through the ILP formulation. Our experiments on widely used machine learning workloads show that OptiPIM can generate the optimal mappings within 4 minutes. The optimal mappings generated by OptiPIM are at least 1.9× faster than those generated by existing heuristic-based and search-based approaches on two emerging PIM accelerators.

Acknowledgments

Many thanks to the anonymous ISCA reviewers for insightful suggestions. This research is supported by the Swiss National Science Foundation (grant number 215747), National Science Foundation (NSF) awards CNS-1730158, ACI-1540112, ACI-1541349, OAC-1826967, OAC-2112167, CNS-2100237, CNS-2120019. This work is also supported by PRISM and CoCoSys, centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by the Defense Advanced Research Projects Agency.

References

[1] Tanner Andrulis, Joel S. Emer, and Vivienne Sze. 2024. CiMLoop: A Flexible, Accurate, and Fast Compute-In-Memory Modeling Tool. In *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS '24)*. Indianapolis, IN, USA, 10–23. https://doi.org/10.1109/ISPASS61541.2024.00012

[2] Aayush Ankit, Izzat El Hajj, Sai Rahul Chalamalasetti, Geoffrey Ndu, Martin Foltin, R. Stanley Williams, Paolo Faraboschi, Wen-mei W Hwu, John Paul Strachan, Kaushik Roy, and Dejan S. Milojicic. 2019. PUMA: A Programmable Ultra-efficient Memristor-based Accelerator for Machine Learning Inference. In *24th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*. Providence, RI, USA, 715–731. https://doi.org/10.1145/3297858.3304049

[3] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, CK Luk, Bert Maher, Yunjie Pan, Christian Puhrsch, Matthias

Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Michael Suo, Phil Tillet, Eikan Wang, Xiaodong Wang, William Wen, Shunting Zhang, Xu Zhao, Keren Zhou, Richard Zou, Ajit Mathews, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. La Jolla, CA, USA, 929–947. https://doi.org/10.1145/3620665.3640366

[4] Prasanth Chatarasi, Hyoukjun Kwon, Angshuman Parashar, Michael Pellauer, Tushar Krishna, and Vivek Sarkar. 2021. Marvel: A Data-Centric Approach for Mapping Deep Learning Operators on Spatial Accelerators. *ACM Trans. Archit. Code Optim.* 19, 1, Article 6 (Dec. 2021). https://doi.org/10.1145/3485137

[5] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*. Carlsbad, CA, USA, 578–594. https://www.usenix.org/conference/osdi18/presentation/chen

[6] Tsu-Wu J. Chou and George E. Collins. 1982. Algorithms for the Solution of Systems of Linear Diophantine Equations. *SIAM J. Comput.* 11, 4 (Nov. 1982), 687–708. https://doi.org/10.1137/0211057 arXiv:https://doi.org/10.1137/0211057

[7] Xiuping Cui, Size Zheng, Tianyu Jia, Le Ye, and Yun Liang. 2023. ARES: A Mapping Framework of DNNs Towards Diverse PIMs with General Abstractions. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD '23)*. San Francisco, CA, USA, 1–9. https://doi.org/10.1109/ICCAD57390.2023.10323777

[8] Shail Dave, Tony Nowatzki, and Aviral Shrivastava. 2024. Explainable-DSE: An Agile and Explainable Exploration of Efficient HW/SW Codesigns of Deep Learning Accelerators Using Bottleneck Analysis. In *28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '23)*. Vancouver, BC, Canada, 87–107. https://doi.org/10.1145/3623278.3624772

[9] Fabrice Devaux. 2019. The true Processing In Memory accelerator . In *2019 IEEE Hot Chips 31 Symposium (HCS '19)*. Cupertino, CA, USA, 1–24. https://doi.org/10.1109/HOTCHIPS.2019.8875680

[10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs.CL] https://arxiv.org/abs/1810.04805

[11] Charles Eckert, Xiaowei Wang, Jingcheng Wang, Arun Subramaniyan, Ravi Iyer, Dennis Sylvester, David Blaaauw, and Reetuparna Das. 2018. Neural Cache: Bit-Serial In-Cache Acceleration of Deep Neural Networks. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA '18)*. Los Angeles, CA, USA, 383–396. https://doi.org/10.1109/ISCA.2018.00040

[12] Fei Gao, Georgios Tziantzioulis, and David Wentzlaff. 2019. ComputeDRAM: In-Memory Compute Using Off-the-Shelf DRAMs. In *52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '52)*. Columbus, OH, USA, 100–113. https://doi.org/10.1145/3352460.3358260

[13] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *IEEE Micro* 44, 3 (May 2024), 33–39. https://doi.org/10.1109/MM.2024.3373763

[14] Aaron Grattafiori et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[15] Gurobi Optimization, LLC. 2023. Gurobi Optimizer Reference Manual. https://www.gurobi.com

[16] Nastaran Hajinazar, Geraldo F. Oliveira, Sven Gregorio, João Dinis Ferreira, Nika Mansouri Ghiasi, Minesh Patel, Mohammed Alser, Saugata Ghose, Juan Gómez-Luna, and Onur Mutlu. 2021. SIMDRAM: a framework for bit-serial SIMD processing using DRAM. In *26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*. Virtual, USA, 329–345. https://doi.org/10.1145/3445814.3446749

[17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR '16)*. Las Vegas, NV, USA, 770–778. https://doi.org/10.1109/CVPR.2016.90

[18] Mingxuan He, Choungki Song, Ilkon Kim, Chunseok Jeong, Seho Kim, Il Park, Mithuna Thottethodi, and T. N. Vijaykumar. 2020. Newton: A DRAM-maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning. In *53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '20)*. Athens, Greece, 372–385. https://doi.org/10.1109/MICRO50266.2020.00040

[19] Kartik Hegde, Po-An Tsai, Sitao Huang, Vikas Chandra, Angshuman Parashar, and Christopher W. Fletcher. 2021. Mind mappings: enabling efficient algorithm-accelerator mapping space search. In *26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*. Virtual, USA, 943–958. https://doi.org/10.1145/3445814.3446762

[20] Charles Hong, Qijing Huang, Grace Dinh, Mahesh Subedar, and Yakun Sophia Shao. 2023. DOSA: Differentiable Model-Based One-Loop Search for DNN Accelerators. In *56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23)*. Toronto, ON, Canada, 209–224. https://doi.org/10.1145/3613424.3623797

[21] Mark Horeni, Pooria Taheri, Po-An Tsai, Angshuman Parashar, Joel Emer, and Siddharth Joshi. 2022. Ruby: Improving Hardware Efficiency for Tensor Algebra Accelerators Through Imperfect Factorization. In *2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS '22)*. Singapore, Singapore, 254–266. https://doi.org/10.1109/ISPASS55109.2022.00039

[22] Huimin Huang, Lanfen Lin, Ruofeng Tong, Hongjie Hu, Qiaowei Zhang, Yutaro Iwamoto, Xianhua Han, Yen-Wei Chen, and Jian Wu. 2020. UNet 3+: A Full-Scale Connected UNet for Medical Image Segmentation. In *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP '20)*. Barcelona, Spain, 1055–1059. https://doi.org/10.1109/ICASSP40776.2020.9053405

[23] Qijing Huang, Minwoo Kang, Grace Dinh, Thomas Norell, Aravind Kalaiah, James Demmel, John Wawrzynek, and Yakun Sophia Shao. 2021. CoSA: Scheduling by Constrained Optimization for Spatial Accelerators. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA '21)*. Virtual, Spain, 554–566. https://doi.org/10.1109/ISCA52012.2021.00050

[24] Bongjoon Hyun, Taehun Kim, Dongjae Lee, and Minsoo Rhu. 2024. Pathfinding Future PIM Architectures by Demystifying a Commercial PIM Technology. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA '24)*. Edinburgh, UK, 263–279. https://doi.org/10.1109/HPCA57654.2024.00029

[25] Mohsen Imani, Saransh Gupta, Yeseong Kim, and Tajana Rosing. 2019. FloatPIM: in-memory acceleration of deep neural network training with high precision. In *46th International Symposium on Computer Architecture (ISCA '19)*. Phoenix, Arizona, 802–815. https://doi.org/10.1145/3307650.3322237

[26] Mingu Kang, Sujan K. Gonugondla, and Naresh R. Shanbhag. 2020. Deep In-Memory Architectures in SRAM: An Analog Approach to Approximate Computing. *Proc. IEEE* 108, 12 (Dec. 2020), 2251–2275. https://doi.org/10.1109/JPROC.2020.3034117

[27] Sheng-Chun Kao and Tushar Krishna. 2020. GAMMA: automating the HW mapping of DNN models on accelerators via genetic algorithm. In *39th International Conference on Computer-Aided Design (ICCAD '20)*. Virtual Event, USA, Article 44, 9 pages. https://doi.org/10.1145/3400302.3415639

[28] Sheng-Chun Kao, Michael Pellauer, Angshuman Parashar, and Tushar Krishna. 2022. DiGamma: Domain-aware Genetic Algorithm for HW-Mapping Co-optimization for DNN Accelerators. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE '22)*. Antwerp, Belgium, 232–237. https://doi.org/10.23919/DATE54114.2022.9774568

[29] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2017. ImageNet classification with deep convolutional neural networks. *Commun. ACM* 60, 6 (May 2017), 84–90. https://doi.org/10.1145/3065386

[30] Hyoukjun Kwon, Prasanth Chatarasi, Michael Pellauer, Angshuman Parashar, Vivek Sarkar, and Tushar Krishna. 2019. Understanding Reuse, Performance, and Hardware Cost of DNN Dataflow: A Data-Centric Approach. In *52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '52)*. Columbus, OH, USA, 754–768. https://doi.org/10.1145/3352460.3358252

[31] Hyoukjun Kwon, Ananda Samajdar, and Tushar Krishna. 2018. MAERI: Enabling Flexible Dataflow Mapping over DNN Accelerators via Reconfigurable Interconnects. In *23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '18)*. Williamsburg, VA, USA, 461–475. https://doi.org/10.1145/3173162.3173176

[32] Yongkee Kwon, Kornijcuk Vladimir, Nahsung Kim, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Guhyun Kim, Byeongju An, Jeongbin Kim, Jaewook Lee, Ilkon Kim, Jaehan Park, Chanwook Park, Yosub Song, Byeongsu Yang, Hyungdeok Lee, Seho Kim, Daehan Kwon, Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gimoon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, Junyeol Jeon, Myeongjun Lee, Minyoung Shin, Minhwan Shin, Jaekyung Cha, Changson Jung, Kijoon Chang, Chunseok Jeong, Euicheol Lim, Il Park, Junhyun Chun, and Sk Hynix. 2022. System Architecture and Software Stack for GDDR6-AiM. In *2022 IEEE Hot Chips 34 Symposium (HCS '22)*. Cupertino, CA, USA, 1–25. https://doi.org/10.1109/HCS55958.2022.9895629

[33] Young-Cheon Kwon, Suk Han Lee, Jaehoon Lee, Sang-Hyuk Kwon, Je Min Ryu, Jong-Pil Son, O Seongil, Hak-Soo Yu, Haesuk Lee, Soo Young Kim, Youngmin Cho, Jin Guk Kim, Jongyoon Choi, Hyun-Sung Shin, Jin Kim, BengSeng Phuah, HyoungMin Kim, Myeong Jun Song, Ahn Choi, Daeho Kim, SooYoung Kim, Eun-Bong Kim, David Wang, Shinhaeng Kang, Yuhwan Ro, Seungwoo Seo, JoonHo Song, Jaeyoun Youn, Kyomin Sohn, and Nam Sung Kim. 2021. 25.4 A 20nm 6GB Function-In-Memory DRAM, Based on HBM2 with a 1.2TFLOPS Programmable Computing Unit Using Bank-Level Parallelism, for Machine Learning Applications. In *2021 IEEE International Solid-State Circuits Conference (ISSCC '21)*, Vol. 64. San Francisco, CA, USA, 350–352. https://doi.org/10.1109/ISSCC42613.2021.9365862

[34] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO '21)*. Seoul, Korea (South), 2–14. https://doi.org/10.1109/CGO51591.2021.9370308

[35] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyounghwan Lim, Hyunsung Shin, Jinhyun Kim, O Seongil, Anand Iyer, David Wang, Kyomin Sohn, and Nam Sung Kim. 2021. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology : Industrial Product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA '21)*. Valencia, Spain, 43–56. https://doi.org/10.1109/ISCA52012.2021.00013

[36] Cong Li, Zhe Zhou, Yang Wang, Fan Yang, Ting Cao, Mao Yang, Yun Liang, and Guangyu Sun. 2024. PIM-DL: Expanding the Applicability of Commodity DRAM-PIMs for Deep Learning via Algorithm-System Co-Optimization. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. La Jolla, CA, USA, 879–896. https://doi.org/10.1145/3620665.3640376

[37] Cong Li, Zhe Zhou, Size Zheng, Jiaxi Zhang, Yun Liang, and Guangyu Sun. 2024. SpecPIM: Accelerating Speculative Inference on PIM-Enabled System via Architecture-Dataflow Co-Exploration. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '24)*. La Jolla, CA, USA, 950–965. https://doi.org/10.1145/3620666.3651352

[38] Shuangchen Li, Dimin Niu, Krishna T. Malladi, Hongzhong Zheng, Bob Brennan, and Yuan Xie. 2017. DRISA: a DRAM-based Reconfigurable In-Situ Accelerator. In *50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '17)*. Cambridge, Massachusetts, 288–301. https://doi.org/10.1145/3123939.3123977

[39] LLVM. [n. d.]. *Torch-MLIR*. https://github.com/llvm/torch-mlir

[40] Liqiang Lu, Naiqing Guan, Yuyue Wang, Liancheng Jia, Zizhang Luo, Jieming Yin, Jason Cong, and Yun Liang. 2021. TENET: A Framework for Modeling Tensor Dataflow Based on Relation-centric Notation. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA '21)*. Virtual, Spain, 720–733. https://doi.org/10.1109/ISCA52012.2021.00062

[41] Haocong Luo, Yahya Can Tuğrul, F. Nisa Bostancı, Ataberk Olgun, A. Giray Yağlıkçı, and Onur Mutlu. 2024. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator. *IEEE Computer Architecture Letters* 23, 1 (Jan. 2024), 112–116. https://doi.org/10.1109/LCA.2023.3333759

[42] Wei Niu, Gagan Agrawal, and Bin Ren. 2024. SoD2: Statically Optimizing Dynamic Deep Neural Network Execution. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '24)*. La Jolla, CA, USA, 386–400. https://doi.org/10.1145/3617232.3624869

[43] NVIDIA. [n. d.]. TensorRT: High-Performance Deep Learning Inference Optimizer and Runtime. https://github.com/NVIDIA/TensorRT Accessed: 2025-02-16.

[44] Chi-Sung Oh, Ki Chul Chun, Young-Yong Byun, Yong-Ki Kim, So-Young Kim, Yesin Ryu, Jaewon Park, Sinho Kim, Sanguhn Cha, Donghak Shin, Jungyu Lee, Jong-Pil Son, Byung-Kyu Ho, Seong-Jin Cho, Beomyong Kil, Sungoh Ahn, Baekmin Lim, Yongsik Park, Kijun Lee, Myung-Kyu Lee, Seungduk Baek, Junyong Noh, Jae-Wook Lee, Seungseob Lee, Sooyoung Kim, Botak Lim, Seouk-Kyu Choi, Jin-Guk Kim, Hye-In Choi, Hyuk-Jun Kwon, Jun Jin Kong, Kyomin Sohn, Nam Sung Kim, Kwang-Il Park, and Jung-Bae Lee. 2020. 22.1 A 1.1V 16GB 640GB/s HBM2E DRAM with a Data-Bus Window-Extension Technique and a Synergetic On-Die ECC Scheme. In *2020 IEEE International Solid-State Circuits Conference (ISSCC '20)*. San Francisco, CA, USA, 330–332. https://doi.org/10.1109/ISSCC19947.2020.9063110

[45] Geraldo F. Oliveira, Ataberk Olgun, Abdullah Giray Yağlıkçı, F. Nisa Bostancı, Juan Gómez-Luna, Saugata Ghose, and Onur Mutlu. 2024. MIMDRAM: An End-to-End Processing-Using-DRAM System for High-Throughput, Energy-Efficient and Programmer-Transparent Multiple-Instruction Multiple-Data Computing. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA '24)*. Edinburgh, UK, 186–203. https://doi.org/10.1109/HPCA57654.2024.00024

[46] GUROBI OPTIMIZATION. 2025. Gurobi Academic License Program: Free Licenses for Academics | Gurobi - Gurobi Optimization. https://www.gurobi.com/academia/academic-program-and-licenses/. Accessed: 2025.

[47] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucek Khailany, Stephen W. Keckler, and Joel Emer. 2019. Timeloop: A Systematic Approach to DNN Accelerator Evaluation. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS '19)*. Madison, WI, USA, 304–315. https://doi.org/10.1109/ISPASS.2019.00042

[48] Louis-Noël Pouchet. 2012. *Polybench: The polyhedral benchmark suite*. https://web.cs.ucla.edu/~pouchet/software/polybench/

[49] Jorge L. Ramírez Alfonsín. 2005. *The Diophantine Frobenius Problem*. Oxford University Press. https://doi.org/10.1093/acprof:oso/9780198568209.001.0001

[50] RDson. 2024. Llama-3-14B-Instruct-v1. https://huggingface.co/RDson/Llama-3-14B-Instruct-v1. Accessed: 2024.

[51] Kenneth H. Rosen. 2002. *Discrete Mathematics and Its Applications* (5th ed.). McGraw-Hill Higher Education.

[52] Enrico Russo, Maurizio Palesi, Giuseppe Ascia, Davide Patti, Salvatore Monteleone, and Vincenzo Catania. 2023. Memory-Aware DNN Algorithm-Hardware Mapping via Integer Linear Programming. In *20th ACM International Conference on Computing Frontiers (CF '23)*. Bologna, Italy, 134–143. https://doi.org/10.1145/3587135.3592206

[53] Amit Sabne. 2020. XLA : Compiling Machine Learning for Peak Performance.

[54] SAITPublic. 2025. PIMSimulator. https://github.com/SAITPublic/PIMSimulator Accessed: 2025-02-16.

[55] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv:1409.1556 [cs.CV] https://arxiv.org/abs/1409.1556

[56] Linghao Song, Xuehai Qian, Hai Li, and Yiran Chen. 2017. PipeLayer: A Pipelined ReRAM-Based Accelerator for Deep Learning. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA '17)*. Austin, TX, USA, 541–552. https://doi.org/10.1109/HPCA.2017.55

[57] Xiaotian Sun, Xinyu Wang, Wanqian Li, Lei Wang, Yinhe Han, and Xiaoming Chen. 2023. PIMCOMP: A Universal Compilation Framework for Crossbar-based PIM DNN Accelerators. In *60th ACM/IEEE Design Automation Conference (DAC '23)*. San Francisco, CA, USA, 1–6. https://doi.org/10.1109/DAC56929.2023.10247928

[58] Zhanhong Tan, Zijian Zhu, and Kaisheng Ma. 2024. Cocco: Hardware-Mapping Co-Exploration towards Memory Capacity-Communication Optimization. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '24)*. La Jolla, CA, USA, 69–84. https://doi.org/10.1145/3617232.3624865

[59] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL '19)*. Phoenix, AZ, USA, 10–19. https://doi.org/10.1145/3315508.3329973

[60] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S. Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. 2018. Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions. arXiv:1802.04730 [cs.PL] https://arxiv.org/abs/1802.04730

[61] Jie Wang and Jason Cong. 2021. Search for Optimal Systolic Arrays: A Comprehensive Automated Exploration Framework and Lessons Learned. arXiv:2111.14252 [cs.AR] https://arxiv.org/abs/2111.14252

[62] Junpeng Wang, Mengke Ge, Bo Ding, Qi Xu, Song Chen, and Yi Kang. 2024. NicePIM: Design Space Exploration for Processing-In-Memory DNN Accelerators With 3-D Stacked-DRAM. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 5 (May 2024), 1456–1469. https://doi.org/10.1109/TCAD.2023.3342605

[63] Yannan Nellie Wu, Po-An Tsai, Angshuman Parashar, Vivienne Sze, and Joel S. Emer. 2022. Sparseloop: An Analytical Approach To Sparse Tensor Accelerator Modeling. In *55th IEEE/ACM International Symposium on Microarchitecture (MICRO '22)*. Chicago, USA, 1377–1395. https://doi.org/10.1109/MICRO56248.2022.00096

[64] Xuan Yang, Mingyu Gao, Qiaoyi Liu, Jeff Setter, Jing Pu, Ankita Nayak, Steven Bell, Kaidi Cao, Heonjae Ha, Priyanka Raina, Christos Kozyrakis, and Mark Horowitz. 2020. Interstellar: Using Halide's Scheduling Language to Analyze DNN Accelerators. In *25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '20)*. Lausanne, Switzerland, 369–383. https://doi.org/10.1145/3373376.3378514

[65] Minxuan Zhou, Guoyang Chen, Mohsen Imani, Saransh Gupta, Weifeng Zhang, and Tajana Rosing. 2021. PIM-DL: Boosting DNN Inference on Digital Processing In-Memory Architectures via Data Layout Optimizations. In *30th International Conference on Parallel Architectures and Compilation Techniques (PACT '21)*. Atlanta, GA, USA, 1–1. https://doi.org/10.1109/PACT52795.2021.00021

[66] Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2022. TransPIM: A Memory-based Acceleration via Software-Hardware Co-Design for Transformer. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA '22)*. Seoul, Korea (South), 1071–1085. https://doi.org/10.1109/HPCA53966.2022.00082

A Artifact Appendix

A.1 Abstract

In this document, we illustrate the detailed steps of installing and running the artifact of OptiPIM for reproducing the key experimental results on analytical model validation (Figure 9), overall performance comparison (Figure 10), multi-layer strategy evaluation (Figure 14) and evaluation on large language models (Figure 15). We have provided our MLIR-based compilation and optimization infrastructure, which can transform a PyTorch program to a nested-loop representation, achieving the optimal performance on one of two PIM architectures: SIMDRAM [16] and HBM-PIM [35]. Our MLIR infrastructure exploits Gurobi [15] for MILP optimization.

Therefore, this document also includes the steps for applying and configuring the Gurobi license. We have also released our PIM simulator, which modifies Ramulator2 [41], and supports cycle-accurate simulation of SIMDRAM and HBM-PIM for nested-loop tensor operators (e.g., convolution, GEMM, etc.). The simulator can directly use the output nested loop of the MLIR infrastructure as input, and generate concrete PIM instructions by conducting induction variable analysis on the nested loop representation. We note that the induction variable analysis and code generation can be integrated into our MLIR infrastructure for implementing a fully functional compiler. We leave this implementation for future release.

A.2 Artifact check-list (meta-information)

- **Program:** C++, Python 3.11
- **Compilation:** g++ ≥ 11 and clang++ ≥ 14
- **Software:** Gurobi [15], MLIR [34]
- **Model:** AlexNet [29], ResNet [17], UNet [22], VGG [55], Bert [10], Llama 3 7B [14], 14B [50]
- **Output:** CSV and PDF files corresponding to Figure 9, 10 14, 15.
- **Experiments:** Compilation, optimization, and PIM trace generation and simulation.
- **How much disk space required?** Minimum 20GB (depending on the number of processeses used).
- **How much time is needed to complete experiments?** Approximate 12 hours on a server (160 Intel Xeon CPU Max 9460 cores) for experiments except the analytical model validation (Figure 9). The analytical validation can take up to 24 hours.
- **Publicly available?** Available on
  - Github: https://github.com/ETHZ-DYNAMO/OptiPIM.
  - Zenodo: https://doi.org/10.5281/zenodo.15103915.
- **Code licenses (if publicly available)?** MIT License
- **Workflow automation framework?** All steps are fully automated with multiple automation scripts and a unified Makefile.

A.3 Description

This artifact contains the source codes of our compilation infrastructure, problem formulation and optimization, cycle-accurate simulator, and automation scripts to run experiments for Figure 9, 10 14, and 15. This artifact also provides the baseline mapping strategies (ASIC [23]) incorporated with our compilation infrastructure for performance comparison. We note that the PIM-DL [65] baseline exploits pseudo-exhaustive (random) search on the mapping, which requires significant runtime (100 hours). Therefore, we provide our pre-searched results in the repository. The provided scripts can automatically generate all figures for reproduction.

A.3.1 How to access. We have archived the code on Zonodo, which supports the download function for a ZIP file of this artifact. The *unzip* command is required after downloading the *OptiPIM.zip* file:

```
$ unzip OptiPIM.zip
```

A.3.2 Software dependencies. This artifact requires Gurobi [15], Python 3.11, g++ ≥ 11 or clang++ ≥ 14 and MLIR [34].

A.3.3 Models. This artifact evaluates several neural network models including AlexNet [29], ResNet [17], UNet [22], VGG [55], Bert [10], Llama 3 7B [14], 14B [50]. Models are acquired from torchvision [3] and HuggingFace, and are then passed to the frontend of our MLIR infrastructure (i.e., Torch-MLIR [39]). Our MLIR infrastructure extracts all convolution and fully-connected layers and passes them to the lowering and optimization modules.

A.4 Installation

To successfully build and run these scripts, please ensure you have (More detailed instructions can be found in the `README.md`):

A.4.1 Libraries Dependencies. Most of our dependencies are provided as standard packages on most Linux distributions. OptiPIM needs a working C/C++ toolchain (compiler, linker), cmake and ninja for building the project, Python (3.11), and standard command-line tools like git.

```
$ sudo apt-get update
$ sudo apt-get install -y clang lld ccache
    cmake ninja-build python3 python3-venv
    libboost-regex-dev git curl gzip
    libreadline-dev unzip
```

A.4.2 Gurobi. Gurobi [15] offers free academic license [46]. Please follow more detailed instructions in the *README* to install your Gurobi license. In addition to adding Gurobi to `$PATH`, OptiPIM's CMake requires the `GUROBI_HOME` environment variable to find headers and libraries:

```
$ export GUROBI_HOME="<your_gurobi_path>/
    gurobiXXXX/linux64"
$ export PATH="${PATH}:${GUROBI_HOME}/bin"
$ export LD_LIBRARY_PATH="$LD_LIBRARY_PATH:
    $GUROBI_HOME/lib"
```

These lines can be added to your shell initialization script.

A.4.3 Python. For a clean, isolated setup, we recommend creating a dedicated Python virtual environment before running any experiments or installing dependencies:

Step 1: Ensure Python 3.11 is installed. Other versions of Python may also work but we have not conducted exhaustive testing.

```
$ python3 --version
```

Step 2: Create a virtual environment and install all dependencies:

```
$ make python_env
```

Step 3: Activate the virtual environment before running any Python scripts:

```
$ source optipim_env/bin/activate
```

Once activated, the shell prompt should indicate the `optipim_env` environment.

Make sure all these dependencies are properly installed before proceeding.

A.4.4 Compile OptiPIM. The Makefile provides targets for all major steps. Below is a rundown of these primary commands:

```
$ make compile_optipim
```

This command conducts the following steps: **1**; Clone and build the LLVM project. **2**; Build the main OptiPIM executable and libraries.

A.5 Sample Test of OptiPIM

To verify your installation and that everything is working correctly, you can run:

```
$ make test
```

This command invokes `test.sh`, which runs a brief sanity check on the OptiPIM setup. The test run is a minimal example to ensure the core functionalities are in place. For full experiments as used in the paper, please generate the figures as described below.

A.6 Experiment workflow

A.6.1 Note on Experiment Duration. Some of these experiments can take a very long time to complete (especially for Figures 9 and 14). To help mitigate this, we have provided pre-parsed results in both CSV and JSON formats. If you prefer not to wait for the full experiments to finish, you may run them partially or skip them altogether and compare your partial outputs against these provided results. This approach ensures you can still reproduce and examine the final figures and trends without incurring the full runtime cost. Each key figure (9, 10, 14, and 15) has a dedicated Makefile target to automate the entire process. Running these will produce the corresponding PDF in the `exp_results/<figure>` directory.

The figures you generate may look slightly different from those in the paper. For Figure 9, we use a large number of randomly generated design points to verify the analytical model, leading to some run-to-run variation. For the other figures, we employ multi-threading to handle different design points in parallel, which can introduce minor discrepancies in the final results. Despite these differences, the overall trends remain consistent with those reported in the paper.

A.6.2 Figure 9. To generate Figure 9, run:

```
$ make fig9
```

This command will: ① build the cycle-accurate simulator, ② randomly generate all simulation input files from all benchmarks, and ③ cross-verify results from cycle accurate simulation and our analytical model. This will produce `fig9.pdf` under `exp_results/fig9`.

Please note that we have provided pre-extracted data files from our experiments (`validation_hbmpim.csv` etc.) in `exp_results/fig9` for convenience. If you simply want to view the final validation figure without re-running the entire experiment, you can execute the plotting script ($validation/correlation_2.py$) directly. However, please be aware that running `make fig9` from scratch will overwrite these pre-extracted files.

A.6.3 Figure 10. To generate Figure 10, run:

```
$ make fig10
```

This command will: ① Generate all needed input files to run OptiPIM, ② Invoke scripts to run desired experiments, ③ Parse all results and generate csv files in `exp_results/fig10` and ④ Generate `fig10.pdf` under `exp_results/fig10/`.

A.6.4 Figure 14. To generate Figure 14, run:

```
$ make fig14
```

This command will: ① Generate all needed MLIR files. ② Run dynamatic programming algorithm to get the desired mapping strategies for different hardware system configurations. ③ Produces `fig14.pdf` under `exp_results/fig14/`.

Please note that the complete experiments for all configurations may take a very long time, thus, A pre-extracted results file (exp_results/fig14/all_layer_results.json) is provided for convenience. Any newly reproduced results will be stored in file ended with `_ae.pdf`. If you only want to generate the final figure without re-running the full experiment, please check the instructions in *README*.

A.6.5 Figure 15. To generate Figure 15, run:

```
$ make fig15
```

This command will: ① Generate all needed MLIR files. ② Run dynamatic programming algorithm to get the desired mapping strategies for different configurations of the LLM. ③ Produces `fig15.pdf` under `exp_results/fig15/`.

A.7 Evaluation and expected results

All evaluation results, including the .CSV files and .PDF figures, will be located in the folders corresponding to the figures, as specified in Appendix A.6. The expected results are figures shown in this paper (Figure 9, 10 14, 15).